Real-Time System Optimization for ROS 2: Scheduling and Communication

PENG, Yifan

Student ID: 225040521

A Thesis Submitted in

CSC6032 Advanced Operating System

School of Data Science

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

The Robot Operating System 2 (ROS 2) has emerged as the de facto standard middleware for modern autonomous robotic systems. However, its widespread deployment in safety-critical applications and resource-constrained embedded platforms is severely limited by three fundamental real-time bottlenecks: non-deterministic user-space scheduling, excessive inter-process communication (IPC) overhead, and the disconnection between static computational policies and dynamic physical safety requirements.

This thesis presents a systematic review of ROS 2 real-time optimization research, focusing on three representative works that form a cohesive, progressive trajectory across three interconnected layers: formal performance modeling, architectural communication optimization, and kernel-level deterministic scheduling. The central argument is that ROS 2 real-time research has evolved along a clear logical pathway: from empirical black-box testing to rigorous mathematical formalization of scheduling flaws, from multi-process isolation to single-process component composition for efficiency, and finally from static worst-case provisioning to physics-informed adaptive scheduling for safety.

At the theoretical foundation lies a seminal work establishing the Compositional Performance Analysis (CPA) framework for the ROS 2 Executor, mathematically proving that the polling-point and snapshot mechanism causes unconditional priority inversion. Building on this diagnosis, an architectural mitigation strategy demonstrates that Node Composition and zero-copy intra-process communication eliminate kernel-level serialization and copying costs, achieving constant-time message passing regardless of payload size. To address the root cause of non-determinism, a state-of-the-art kernel-level solution introduces a fully preemptable EDF executor and a novel Physics-Informed Mixed-Criticality scheduling framework, resolving both the non-preemption problem and the resource waste inherent in static worst-case design.

Through a unified comparative framework, this thesis evaluates each work's problem formulation, technical contributions, experimental methodology, strengths, and limitations, while articulating the complementary interplay between the three paradigms. It concludes that the field is converging toward cross-layer, physics-aware deterministic real-time systems, yet open challenges remain in heterogeneous hardware scheduling, distributed real-time determinism over wireless networks, and the trade-off between zero-copy efficiency and cybersecurity.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Research Background

The dawn of the era of autonomous and intelligent robotics has witnessed a fundamental shift in how complex mechanical systems perceive, reason, and interact with the physical world. Modern robotic platforms, ranging from self-driving vehicles to industrial collaborative manipulators, are equipped with an increasingly sophisticated array of sensors and high-performance algorithms for perception, localization, and motion planning. To manage the immense complexity of these systems, developers require a robust framework that abstracts low-level hardware details and provides a scalable environment for algorithm integration.

The Robot Operating System (ROS) has emerged as the de facto standard middleware framework for this purpose. Often described as "Lego for robots," ROS provides a modular ecosystem that enables developers to compose disparate software components into a unified system through a standardized communication interface. In the ROS paradigm, functional units are encapsulated as "Nodes," where sensors act as publishers of data and algorithms act as subscribers. This abstraction allows researchers to focus on high-level intelligence rather than the intricacies of inter-process communication (IPC) and device drivers.

Historically, the first generation of ROS (ROS 1) played a pivotal role in the "Algorithm Boom," where researchers utilized powerful workstations to develop complex SLAM and navigation stacks. However, ROS 1 was primarily designed for academic research and lacked the robustness required for commercial, safety-critical applications. As the industry transitioned into a "Commercial Explosion" phase, robots moved from laboratory environments to real-world markets, often running on cost-effective, resource-constrained embedded platforms. This shift necessitated the development of ROS 2, which was re-engineered from the ground up to support real-time requirements, multi-robot systems, and small-scale embedded hardware by adopting the Data Distribution Service (DDS) as its communication backbone.

From an operating system perspective, ROS 2 introduces a sophisticated execution model

that maps robotic abstractions to OS-level primitives. While a traditional OS manages hardware resources like CPU and memory, ROS 2 manages robotic resources such as sensors and actuators. In this architecture, a ROS Node is analogous to an OS process, and the internal logic units, or "callbacks," are analogous to threads. To bridge the gap between middleware logic and OS scheduling, ROS 2 introduces the "Executor"—a user-level scheduler responsible for determining the execution order of callbacks within an OS thread.

However, as robotics applications become more safety-critical, the end-to-end latency —defined as the total time elapsed from sensor data acquisition to final motor actuation— becomes the most vital metric for system success. In domains such as autonomous racing (e.g., F1Tenth), a delay exceeding strict temporal bounds can lead to catastrophic collisions, highlighting a disconnect between physical safety and traditional computational scheduling. Consequently, optimizing the scheduling and communication mechanisms of ROS 2 to ensure deterministic real-time performance has become a paramount research challenge in the field of advanced operating systems.

1.2 Three Existing Problems of ROS 2

Despite its widespread adoption, ROS 2 faces several critical bottlenecks that hinder its performance in safety-critical and resource-constrained robotic environments. Through an integrated analysis of current research, these challenges can be categorized into three fundamental dimensions:

- **Non-Deterministic User-Space Scheduling:** The default ROS 2 Executor employs a "snapshot-based" scheduling mechanism at its polling points. This design introduces significant priority inversion and head-of-line blocking, as high-priority callbacks must wait for the entire set of previously snapped low-priority tasks to complete. Such a non-preemptive nature in user space undermines the timing guarantees provided by the underlying real-time operating system (RTOS).

- **Architectural Communication Overhead:** The traditional multi-process model in ROS 2 ensures isolation but imposes heavy Inter-Process Communication (IPC) costs. For high-bandwidth data such as 4K images or LiDAR point clouds, the $O(N)$ complexity of serialization and kernel-space copying leads to excessive CPU and memory saturation. This is particularly problematic for commercial robotic platforms utilizing embedded ARM chips with limited computational power.

- **Disconnection from Physical Reality:** Existing ROS 2 scheduling policies are often "blind" to the robot's physical environment. They typically rely on static worst-case execution time (WCET) assumptions, leading to over-provisioning of resources. Without the ability to dynamically adapt scheduling parameters based on physical

safety metrics (e.g., proximity to obstacles), the system fails to achieve an optimal tradeoff between safety and computational efficiency.

1.3 Thesis Contributions

This thesis makes four primary contributions to the optimization of real-time performance in ROS 2-based robotic systems, spanning formal analysis, architectural optimization, and kernel-level scheduling:

- **Formal Modeling and Performance Bottleneck Diagnosis.** This thesis provides a rigorous mathematical formalization of the ROS 2 Executor's behavior within resource-reserved environments. By applying Compositional Performance Analysis (CPA), it derives the first set of equations for calculating the Worst-Case Response Time (WCRT) of ROS 2 processing chains. Most importantly, it theoretically exposes the inherent unpredictability caused by the *polling point* (Snapshot Scheduling) mechanism, proving that default priority assignments are essentially ineffective for strict latency guarantees.

- **Architectural Optimization via Node Composition and Zero-Copy IPC.** To address the significant spatial and communication overhead in multi-process robotic systems, this thesis introduces an architectural transition to a single-process Component Container model. It implements an Intra-Process Communication (IPC) mechanism utilizing zero-copy pointer passing, effectively bypassing kernel-level serialization and copying costs. Furthermore, it explores isolated execution through dedicated OS threads per node to ensure superior responsiveness.

- **Kernel-Level Preemptive Scheduling and Physics-Informed Criticality.** This thesis proposes a fundamental scheduling fix by designing a Preemptable Executor that maps individual callbacks to unique OS threads, enabling true kernel-level preemption via the Linux `SCHED_DEADLINE` policy. Additionally, it introduces a *Physics-Informed Mixed-Criticality* policy that dynamically adjusts task deadlines and drops non-critical tasks based on the robot's physical state (e.g., distance to obstacles), achieving a robust balance between safety and computational efficiency.

- **Systematic Synthesis of Optimization Paradigms.** Through a cross-layer synthesis, this thesis categorizes and compares the evolution of these optimization strategies. It articulates a complete "diagnostic-to-treatment" pathway: from identifying the "Priority Inversion" illness in the middleware logic, to prescribing architectural treatments for communication overhead, and finally implementing kernel-level interventions for absolute timing safety.

1.4 Methodological Posture

The methodological posture of this thesis is an interpretive and critical synthetic review, rather than purely empirical research or independent system re-implementation. No independent replication of the three core systems, re-running of benchmark experiments, or hardware-in-the-loop validation on physical robotic platforms is conducted. Instead, the analysis is built on rigorous in-depth reading of the primary focal papers, systematic inspection of their formal mathematical models and system design principles, side-by-side comparison of their experimental protocols and reported results, and critical evaluation of their technical claims under a unified comparative framework.

The core strength of this work lies in the structured synthesis of the evolutionary trajectory of ROS 2 real-time optimization, critical cross-paradigm comparison of the three representative works, clear positioning of each method within the broader real-time systems literature, and systematic analysis of deployment trade-offs for industrial robotic applications. At the same time, its inherent boundaries are explicitly defined. All quantitative performance analysis and numerical conclusions are derived from the experimental settings and results reported in the original focal papers, with no independent empirical validation conducted, so the validity of all quantitative claims is inherently dependent on the rigor and reproducibility of the original experimental designs. Direct cross-paper numerical comparisons must be interpreted with caution, as the three focal papers differ in test platforms, experimental scenarios, benchmark metrics, and candidate system configurations; such comparisons are used only to illustrate relative optimization effects rather than to establish an absolute performance ranking across systems. Finally, the scope of this thesis is deliberately bounded, focusing on three core layers of single-node ROS 2 real-time optimization—formal performance modeling and scheduling flaw diagnosis, architecture-level optimization via Node Composition, and kernel-level preemptive scheduling with physics-informed mixed-criticality control—without attempting an exhaustive survey of all ROS 2 real-time optimization research or covering adjacent areas including real-time optimization of the underlying DDS communication protocol, global multi-core scheduling, and distributed real-time coordination for multi-robot systems.

1.5 Thesis Organization

The remainder of this dissertation is organized as follows.

Chapter 2 establishes the technical foundations and provides a comprehensive review of related work in real-time scheduling, middleware architecture, and cyber-physical system verification. **Chapter 3** focuses on the formal modeling and analysis of the default ROS 2 Executor. By utilizing Compositional Performance Analysis (CPA), this chapter mathematically quantifies the performance bottlenecks caused by the polling point mechanism, theoretically exposes the flaws in its priority assignment, and empirically validates these

findings through end-to-end latency evaluations. **Chapter 4** introduces an architectural mitigation strategy via Node Composition. This chapter explores the transition from multi-process to single-process containers and benchmarks the impact of zero-copy communication and thread isolation on reducing system overhead across extensive real-world evaluations, including the Nav2 navigation stack. **Chapter 5** proposes a kernel-level solution through a Preemptable EDF Executor combined with Physics-Informed Mixed-Criticality scheduling. It demonstrates how to achieve deterministic performance by mapping callbacks to dedicated threads, and rigorously validates the safety of dynamic mode switching through formal UPPAAL verification and physical F1Tenth racing experiments. **Chapter 6** provides a comprehensive synthesis of the proposed optimization paradigms. It analyzes the logical evolution of these works, identifies critical open challenges for next-generation autonomous systems, and offers a macroscopic perspective on the redefining role of robotic operating systems. Finally, **Chapter 7** concludes the dissertation and outlines potential directions for future research.

Chapter 2

Related Works

The systematic optimization of real-time middleware involves a multi-layered research trajectory, ranging from theoretical response-time modeling to practical architectural redesigns and adaptive scheduling policies. This chapter reviews the historical evolution and current state of research in these domains, providing the necessary context for the optimization paradigms presented in this thesis.

2.1 Real-Time Scheduling and Analysis in Middleware

In the mid-2010s, as real-time systems increasingly relied on multi-core processors and complex communication pipelines, the focus of scheduling analysis shifted toward event-driven task chains. Compositional Performance Analysis (CPA), pioneered by Henia et al. [10], emerged as the dominant framework for analyzing heterogeneous distributed systems, modeling systems as networks of resources and workloads as tasks with dependencies. Building on this foundation, Schlatow and Ernst [19] advanced the state-of-the-art by developing a response-time analysis framework for task chains executing across communicating threads. While this method effectively captured end-to-end latencies under data-driven activation models, it predominantly assumed traditional kernel-level preemptive semantics. Consequently, it lacked the theoretical granularity to accurately model the internal synchronization artifacts and non-deterministic behaviors introduced by user-space middleware schedulers.

As ROS 2 emerged to fill the middleware void for production-grade robotics, early research prioritized empirical performance over formal modeling. For instance, Maruyama et al. [17] conducted a pioneering performance exploration of the ROS 2 framework and its underlying Data Distribution Service (DDS). Their study provided critical empirical insights into the latency overheads associated with the middleware's communication layers. However, this work was heavily experimental; it lacked a rigorous mathematical model capable of providing safe worst-case response time (WCRT) bounds for safety-critical processing chains.

The critical gap between formal real-time analysis and the unique implementation mechanics of ROS 2 was systematically bridged by Casini et al. [6]. They formalized the specific behavior of the ROS 2 Executor within the CPA framework. Their research was the first to mathematically capture the ”snapshot” scheduling logic and polling-point mechanisms inherent in the default executor. By theoretically proving that such user-space mechanisms cause severe priority inversion—and render default priority assignments largely ineffective under reservation-based scheduling—their work established the foundational problem formulation that motivates the architectural and kernel-level optimizations explored in the remainder of this thesis. Subsequent work by Blaß et al. [4] refined this analysis by exploiting starvation freedom and execution-time variance to derive tighter WCRT bounds, while Jiang et al. [12] extended the model to support multi-threaded executors, addressing the limitations of the original single-threaded analysis.

2.2 Inter-Process Communication and Architectural Optimizations

To systematically address the communication and execution bottlenecks in ROS 2, significant research has been dedicated to empirical performance measurement and architectural redesign. Understanding the latency introduced by Inter-Process Communication (IPC) requires robust profiling tools capable of inspecting the middleware without perturbing its real-time behavior. Bédard et al. [3] introduced *ros2_tracing*, a low-overhead framework based on the Linux Trace Toolkit Next Generation (LTTng) that allows detailed instrumentation of a running ROS 2 system. This foundational tracing framework operates seamlessly across the entire ROS 2 software stack to extract low-level execution timings. It was subsequently extended by Bédard et al. [2] to perform complex message flow analysis by associating causally linked events across the ROS 2 computation graph, enabling developers to visualize execution pipelines using tools like Trace Compass.

Building upon these kernel- and user-space tracing capabilities, several application-level evaluation tools emerged to quantify communication delays in realistic autonomous workloads. Li et al. [15] developed *Autoware_Perf*, which leverages the tracing framework to measure crucial metrics such as end-to-end, intra-node, and inter-node latencies within autonomous driving stacks. Similarly, Peng et al. [18] utilized this framework to evaluate scheduling performance against Autoware.Auto, providing a reference architecture assessment for autonomous driving. Further advancing executor analysis, Kuboichi et al. [14] introduced the Chain-Aware ROS Evaluation Tool (CARET), designed to specifically measure and visualize message flows at the application level. While these empirical tools provide deep visibility into system behavior and latency distribution, they primarily act as observational instruments. They focus on measuring the default execution models and do not systematically characterize or resolve the architectural impact of alternative deploy-

ment paradigms, such as node composition or varying communication mechanisms.

To move beyond observation and actively mitigate communication and scheduling overheads, researchers have proposed direct architectural enhancements. Recognizing that communication methods drastically alter system delays, Jiang et al. [12] extended theoretical response-time models to explicitly account for the latency differences between inter-process and intra-process communication within multi-threaded executors. Other efforts have focused on replacing the default ROS 2 executor entirely to circumvent its inherent scheduling overheads. Yang and Azumi [27] proposed a callback group-based executor specifically designed to make ROS 2 more suitable for strict real-time environments, significantly improving determinism by refining callback prioritization. Additionally, Choi et al. [7] developed PiCAS, a priority-driven, chain-aware scheduler that actively considers the entire callback execution chain to establish tighter bounds on both average and worst-case latencies.

Despite these advances in executor design, they remain largely bounded by the fundamental limitations of the multi-process deployment paradigm. Optimizing the underlying Inter-Process Communication architecture—specifically by bypassing the high computational costs of message serialization and DDS network traversal—remains a critical factor in achieving true high-performance data processing in robotic systems. This critical gap in the literature strongly motivates the exploration of single-process architectural mitigations via Node Composition, which directly targets the structural inefficiencies of ROS 2 data transport.

2.3 Mixed-Criticality Systems and Kernel-Level Preemption

While architectural optimizations like Node Composition effectively reduce communication latency, they do not inherently guarantee temporal safety under varying physical conditions. The most recent frontier in robotics research focuses on bridging the critical gap between computational timing and physical safety, often by employing dual-controller architectures and mixed-criticality scheduling. The foundational Simplex architecture, introduced by Seto et al. [20] and Sha [21], established the paradigm of pairing an unverified high-performance controller with a mathematically verified high-assurance fallback controller. Extending this concept to modern autonomous systems, Genin et al. [9] utilized formal methods to identify unsafe state spaces, demonstrating that dynamically switching to a safe fallback controller enables the system to recover from otherwise unrecoverable physical states.

To support such dynamic mode switching at the scheduling layer, researchers have extensively explored Mixed-Criticality (MC) systems. Following Vestal's seminal formalization of MC models [24], the field has expanded significantly, as comprehensively surveyed

by Burns and Davis [5]. The original MC models have been adapted for both fixed-priority and dynamic-priority scheduling environments, with notable advancements in dynamic-priority handling by Baruah et al. [1] and Ekberg and Yi [8]. However, a fundamental limitation of these traditional MC frameworks is their exclusive reliance on temporal overruns (i.e., when a task exceeds its expected execution time) to trigger mode switches. They generally lack the mechanisms to adapt scheduling parameters based on dynamic variations in the external physical environment.

Simultaneously, the rigorous safety assurance of cyber-physical systems (CPS), particularly autonomous vehicles, has necessitated the integration of advanced formal methods. As emphasized by Wing [26], addressing scenarios with life-critical consequences requires entirely new formal verification techniques. In the domain of autonomous ground vehicles, Kopylov et al. [13] verified a ”safety net” for waypoint navigation controllers using theorem proving. More specifically targeting the F1Tenth racing platforms, Ivanov et al. [11] demonstrated the feasibility of verifying the safety properties of neural network-based controllers. Despite these advancements in verifying the control logic, a disconnect remains between the verified physical safety bounds and the underlying real-time operating system scheduler.

Within the specific context of ROS 2, bridging this disconnect requires precise schedulability analysis and kernel-level predictability. Following the pioneering response-time analysis of the ROS 2 executor by Casini et al. [6], numerous subsequent studies, including those by Tang et al. [22], Blaß et al. [4], and Teper et al. [23], have iteratively refined and improved the worst-case latency bounds for ROS 2 processing chains. While these studies enhanced the theoretical understanding of user-space scheduling, they highlighted the intrinsic difficulties of achieving exact schedulability in the standard ROS 2 middleware. This persistent gap definitively motivates the need for kernel-level preemptive execution combined with a physics-informed mixed-criticality approach, directly inspiring the final optimization paradigm presented in Chapter 5 of this thesis.

2.4 Relationship between three selected core papers

The three core papers selected for this thesis form a cohesive, progressive research trajectory that systematically addresses the real-time performance challenges of ROS 2. Rather than presenting isolated, incremental improvements, these works constitute a complete logical progression: from fundamental problem identification and formal diagnosis, to architectural efficiency mitigation, and finally to kernel-level deterministic control for safety-critical systems.

As visualized in Figure 2.1, the three papers map sequentially to three interconnected layers of ROS 2 real-time optimization. The first paper anchors the *Problem Identification* phase at the modeling layer, laying the theoretical foundation by formalizing the root causes

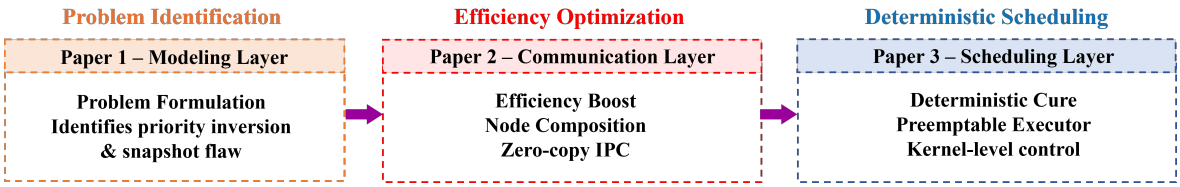

Figure 2.1: Progressive research trajectory of the three core papers: from problem identification to efficiency optimization and deterministic scheduling

of non-determinism in the default ROS 2 executor. Building on this diagnosis, the second paper advances into the *Efficiency Optimization* phase at the communication layer, resolving the architectural overhead bottlenecks of traditional multi-process deployments. Finally, the third paper delivers the *Deterministic Scheduling* solution at the kernel layer, addressing the fundamental scheduling flaws and bridging the gap between computational timing and physical safety.

Table 2.1: Comparative analysis and functional positioning of the three core papers

Paper	**Primary Contribution**	**Optimization Layer**	**Role in the Thesis**
Casini et al. [6]	Formal WCRT analysis of ROS 2 Executor	Modeling Layer	**Problem Formulation:** Identifies priority inversion and snapshot scheduling flaws.
Macenski et al. [16]	Node Composition and Zero-Copy IPC	Communication Layer	**Efficiency Boost:** Minimizes $O(N)$ communication overhead and system resource consumption.
Wilson et al. [25]	Preemptable Executor and Physics-Informed MC	Scheduling Layer	**Deterministic Cure:** Enables kernel-level preemption and physics-aware safety guarantees.

The complementary relationship between these works is rooted in their end-to-end solution to the ROS 2 latency and determinism problem, as summarized in Table 2.1. [6] estab-

lishes the indispensable theoretical foundation by exposing the inherent flaws of the default user-space executor—specifically the ”snapshot” polling mechanism that causes unconditional priority inversion. While this work delivers a rigorous diagnosis of the ”scheduling disease”, it does not propose a structural, deployable cure. [16] addresses the system efficiency bottleneck via Node Composition: by collapsing multiple processes into a single container and enabling zero-copy pointer passing, it drastically reduces end-to-end latency and system resource overhead. However, while Node Composition makes the system ”faster” in average-case performance, it does not inherently fix the non-deterministic scheduling logic of the default executor. Finally, [25] bridges this critical gap by reconstructing the ROS 2 execution mechanism itself. By implementing a fully Preemptable Executor and integrating physical safety metrics into the scheduling policy, it achieves the strict timing determinism and environmental adaptivity required for high-speed safety-critical autonomous systems.

In summary, the evolution from [6] to [16] and [25] maps the complete developmental trajectory of ROS 2 real-time optimization: from empirical black-box testing to formal mathematical diagnosis of middleware behavior, from streamlining inter-process data flow to eliminating architectural overhead, and ultimately to subordinating middleware execution to the native deterministic efficiency of the OS kernel and the safety constraints of the physical world.

Chapter 3

Performance Bottlenecks and Scheduling Flaws

3.1 Motivation

The transition of robotic systems from experimental prototypes to safety-critical autonomous platforms has shifted the focus of system design from average-case throughput to worst-case determinism. In complex robotic applications, such as autonomous racing or industrial collaborative robots, the system's correctness depends not only on the logical results of its computations but also on the physical time at which these results are produced. This temporal requirement is best encapsulated by the concept of end-to-end latency, which tracks the total delay along a functional processing chain—from the moment a sensor perceives the environment to the moment an actuator executes a corresponding command.

Despite the critical nature of these timing constraints, providing rigorous latency guarantees in ROS 2 remains a formidable challenge due to the semantic gap between the operating system (OS) and the middleware. Modern real-time operating systems provide sophisticated resource reservation mechanisms, such as Linux `SCHED_DEADLINE`, which can isolate CPU bandwidth for specific processes. However, ROS 2 introduces a secondary layer of scheduling in user space through the "Executor" component. The Executor is responsible for multiplexing multiple logical units, known as callbacks, onto one or more OS threads.

A fundamental problem arises because the default ROS 2 Executor remains "blind" to the timing requirements of the callbacks it manages. As identified in early performance studies, the Executor utilizes a non-preemptive polling-point mechanism that takes a "snapshot" of ready tasks. Once a snapshot is taken, the Executor processes all tasks in the set sequentially, regardless of their priority or the arrival of new, more urgent events. This behavior can lead to severe priority inversion and head-of-line blocking, effectively neutralizing the timing guarantees provided by the underlying OS-level scheduler.

Empirical testing alone is insufficient for verifying such systems, as it cannot guaran-

tee that the worst-case scenario has been observed. Therefore, there is an urgent need for a formal mathematical framework to model the internal behavior of the ROS 2 Executor. Such a model is essential for deriving Worst-Case Response Time (WCRT) bounds and for providing the formal certificates of safety required for the next generation of autonomous cyber-physical systems. This chapter addresses this need by providing a rigorous analysis of the performance bottlenecks and scheduling flaws inherent in the default ROS 2 architecture.

3.2 System Model and Task Chain Representation

To provide a rigorous timing analysis, this chapter first formalizes the software architecture of ROS 2 into a deterministic real-time scheduling model, building upon the foundational system definition proposed by Casini et al. [3]. As illustrated in Figure 3.1, a ROS 2 system is structured as a hierarchical layered stack. The application layer interacts with the underlying operating system through the ROS client library (rcl) and the ROS Middleware (rmw) interface, which serves as a crucial abstraction layer over various Data Distribution Service (DDS) vendor implementations.

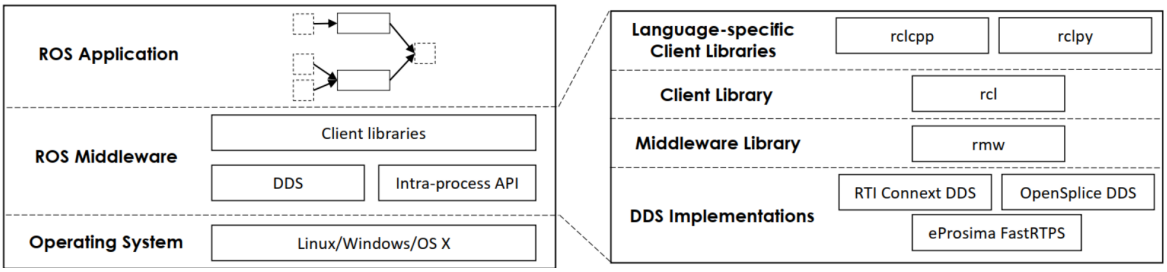

Figure 3.1: Layered structure of ROS 2, showing the mapping from application nodes to OS threads via executors and DDS middleware.

Formally, a ROS 2 system is modeled as a directed acyclic graph (DAG) $D = \{C, E\}$, where $C = \{c_1, c_2, \ldots, c_n\}$ denotes the set of all callbacks—representing the elementary, atomic units of execution within the middleware—and $E \subseteq C \times C$ denotes the set of directed edges encapsulating cause-effect activation relations. Callbacks are rigorously categorized into four distinct types based on their underlying activation triggers:

- C^{tmr}: Timer callbacks, activated periodically by internal system timers.
- C^{sub}: Subscription callbacks, triggered asynchronously by incoming data streams on specific DDS topics.
- C^{srv}: Service callbacks, activated by incoming remote procedure call (RPC) requests.
- C^{clt}: Client callbacks, triggered by the arrival of responses to previously dispatched service requests.

Each callback $c_i \in C$ is mathematically characterized by its worst-case execution time (WCET) e_i and a static priority assignment π_i. To orchestrate the execution of these callbacks, the architecture defines a set of executors $\mathcal{E} = \{E_1, E_2, \dots\}$. During the system initialization phase, each callback is statically assigned to exactly one executor $E_k \in \mathcal{E}$, which inherently runs on a single, dedicated OS-level thread. This formalization focuses exclusively on the standard single-threaded executor, as it remains the most widely deployed and structurally predictable configuration in safety-critical deployments.

In complex cyber-physical applications, end-to-end robotic behaviors (e.g., perception-to-actuation pipelines) are realized through processing chains. These are represented as directed paths within the DAG, where the logical completion of a predecessor callback triggers the activation of its successor. Formally, a processing chain $\gamma^x = (c_s, \dots, c_e)$ is an ordered sequence of callbacks where every adjacent pair forms a valid edge $(c_i, c_{i+1}) \in E$. The overarching objective of this analytical framework is to mathematically bound the end-to-end latency of γ^x, explicitly defined as the maximum physical time elapsed from the initial activation of the source callback c_s to the successful completion of the sink callback c_e.

To guarantee strict temporal isolation and prevent localized computational overloads from cascading across the robotic system, it is assumed that all executor OS threads operate under rigid resource reservations, such as those provided by the Linux `SCHED_DEADLINE` policy. A resource reservation r_k allocated to executor E_k is defined by the tuple (Q_k, P_k), where Q_k represents the maximum CPU budget strictly enforced by the kernel during every replenishment period P_k. Consequently, the absolute processor bandwidth accessible to the executor is bounded by:

$$U_k = \frac{Q_k}{P_k} \tag{3.1}$$

The minimum guaranteed CPU time provisioned by reservation r_k over any arbitrary continuous time interval of length Δ is formalized by the supply-bound function $sbf_k(\Delta)$. This step-wise continuous function serves as the irrefutable foundation for all subsequent worst-case response-time derivations.

3.3 Formalization of the Polling Point and Snapshot Mechanism

A critical semantic mismatch exists between traditional operating systems and the ROS 2 middleware: the default ROS 2 executor implements a user-space scheduling logic that fundamentally contradicts standard kernel-level preemptive principles. Its execution flow is entirely dictated by a polling point mechanism intertwined with snapshot semantics, precisely formalized by the algorithm depicted in Figure 3.2.

When the execution queue is empty, the executor thread blocks, waiting for notifica-

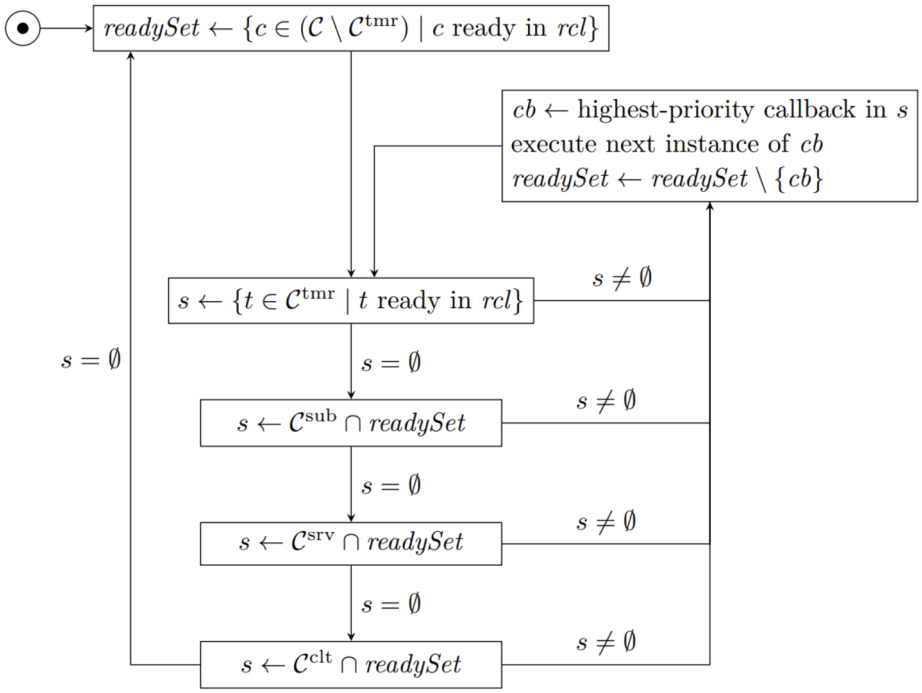

Figure 3.2: The default ROS 2 single-threaded executor scheduling algorithm.

tion of incoming events at the DDS layer. Upon event arrival, the executor awakens and reaches a discrete computational instance formally defined as a *polling point*. At this exact microsecond, the executor invokes internal middleware APIs (e.g., `rcl_wait`) to query for all pending non-timer callbacks, taking a frozen *snapshot* of the system's state. The identified callbacks are then cached into an internal memory collection known as the *ReadySet*. Subsequently, the executor drains this ReadySet by processing the captured callbacks in a strictly hard-coded, fixed-priority sequence: timers are prioritized first, followed sequentially by subscriptions, services, and finally clients. Crucially, the system state is locked; the ReadySet is never updated until it is entirely exhausted. Any newly arriving callback that becomes ready immediately after the snapshot is taken is entirely blind to the executor until the next polling point occurs.

This unique execution model introduces two pathological flaws that severely compromise real-time predictability:

1. **Non-Preemptive Batch Execution**: Once the ReadySet is instantiated, the executor processes all buffered callbacks sequentially to completion. It does not yield the thread nor poll the middleware for newer events, rendering the system effectively non-preemptive at the application layer.

2. **Snapshot-Induced Starvation**: Extremely critical, high-priority callbacks that arrive moments after a polling point are forced to endure the cumulative execution time of the entire current batch of potentially low-priority tasks before they can even be

scheduled.

As illustrated in Figure 3.3, these operational mechanics generate a unique, highly pessimistic interference pattern. Unlike traditional RTOS environments, any target callback in ROS 2 can be subjected to scheduling delays caused by all other callbacks sharing the same executor, largely disregarding user-defined priority assignments.

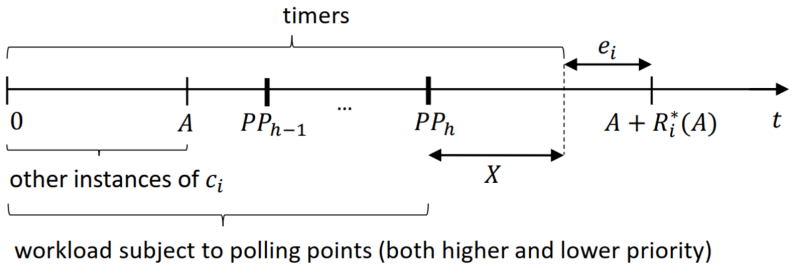

Figure 3.3: Time intervals in which other pp-based callbacks and timers can interfere with a pp-based callback under analysis.

Mathematically, this formalization dictates that two distinct sources of temporal interference affect any target callback c_i:

- **Internal Interference**: The accumulated delay caused by the non-preemptive, sequential execution of all other callbacks captured within the current or preceding ReadySet.
- **Reservation Interference**: The external delay introduced when the underlying OS thread inevitably depletes its CPU budget Q_k and is forcibly suspended by the kernel until the commencement of the next replenishment period P_k.

This structural analysis definitively reveals that the default ROS 2 executor suffers from unconditional, built-in priority inversion—a systemic flaw that cannot be circumvented through mere parameter tuning or priority adjustments.

3.4 Response-Time Analysis via Compositional Performance Analysis

To mathematically quantify the worst-case response time (WCRT) of ROS 2 processing chains and expose the severity of the aforementioned flaws, Casini et al. leverage the Compositional Performance Analysis (CPA) framework. The core analytical principle of CPA, visually synthesized in Figure 3.4, is to evaluate system schedulability as a continuous race between the cumulative CPU execution demand generated by the software workload and the guaranteed CPU supply explicitly provided by the OS-level reservation.

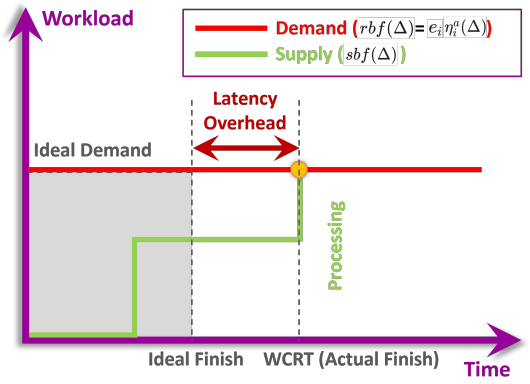

Figure 3.4: Supply vs. Demand modeling using Compositional Performance Analysis (CPA)

Within the CPA methodology, the absolute maximum CPU demand generated by a single callback c_i over any continuous time window of length Δ is formalized by the request-bound function:

$$rbf_i(\Delta) = \eta_i^a(\Delta) \cdot e_i \tag{3.2}$$

where $\eta_i^a(\Delta)$ denotes the activation curve, strictly representing the maximum possible number of instances of c_i that can be released within the interval Δ. Consequently, the aggregated processing demand from an arbitrary set of callbacks Ω is defined as:

$$RBF(\Omega, \Delta) = \sum_{c_i \in \Omega} rbf_i(\Delta) \tag{3.3}$$

The theoretical worst-case response time of a callback occurs at the exact smallest time t where the step-wise cumulative CPU supply successfully accommodates the cumulative CPU demand. For callbacks subjected to the polling point mechanism (denoted as pp-based callbacks, encompassing all subscriptions, services, and clients), Casini et al. prove that the critical worst-case interference scenario transpires when the target callback arrives an infinitesimal fraction of time after a snapshot has been taken. Under these pessimistic conditions, the response-time boundary simplifies to the core mathematical revelation of the analysis:

$$sbf_k\left(A + R_i^*(A)\right) = rbf_i(A+1) + RBF\left(\{\mathcal{C}_k \setminus \{c_i\}\}, A + R_i^*(A) - e_i + 1\right) \tag{3.4}$$

where A represents the release offset of the target callback relative to the commencement of the busy period, and $R_i^*(A)$ is the specific response time for that given offset. The ultimate WCRT of c_i is derived by maximizing $R_i^*(A)$ across all mathematically valid offsets.

As dramatically conceptualized in Figure 3.5, Equation (3) mathematically exposes the fatal vulnerability of the default ROS 2 scheduling paradigm. In a standard preemptive RTOS, a task's demand equation is strictly insulated, experiencing interference solely from

the set of higher-priority tasks. In stark contrast, Equation (3) proves that in ROS 2, a pp-based callback experiences interference from the demand of *all other callbacks* residing in the same executor ($\{\mathcal{C}_k \setminus \{c_i\}\}$), indiscriminately blocking regardless of their designated priority. This provides the definitive mathematical proof that default priority assignments in ROS 2 are intrinsically ineffective for securing worst-case latency guarantees.

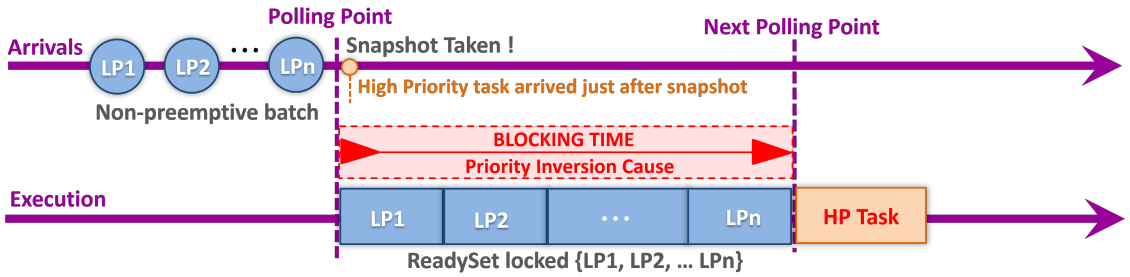

Figure 3.5: Comparison of interference models: standard OS vs. ROS 2 default executor

For the evaluation of complete end-to-end processing chains, Casini et al. expand this analytical model to subchains that remain confined within a single reservation boundary. To compute the total latency of a global chain traversing multiple reservations, the framework sums the bounded response times of each isolated subchain along with the respective communication delays between reservations. This holistic formulation correctly accounts for the real-world ”pay-burst-only

3.5 Evaluation and Results Analysis

To validate the theoretical response-time analysis presented in the previous section, Casini et al. [3] conducted a comprehensive case study on the `move_base` package—the core navigation stack of ROS 2. The experiment focused on the safety-critical processing chain from odometry input to velocity command output, which determines how quickly a robot can react to unexpected obstacles.

3.5.1 Experimental Setup

The evaluation was performed on a standard x86-64 Linux platform with the PREEMPT-RT real-time patch. The `move_base` subsystem was configured to run under the Linux `SCHED_DEADLINE` resource reservation scheduler, which provides guaranteed CPU bandwidth isolation. Two system designs were compared:

1. **Time-driven design**: The local planner runs periodically at a fixed 12.5 Hz rate, regardless of sensor data arrival times

2. **Event-driven design**: The local planner is triggered immediately upon the arrival of new sensor data

All experiments used realistic worst-case execution times (WCETs) and sensor rates obtained from Bosch industrial robotics deployments. The primary metric of interest was the end-to-end worst-case response time (WCRT) of the critical navigation chain.

3.5.2 Effect of Resource Reservation Budget

The first experiment investigated how the CPU budget allocated to the local planner reservation affects the end-to-end latency. The results are shown in Figure 3.6, which plots the predicted WCRT as a function of the reservation budget (expressed as a percentage of a single CPU core).

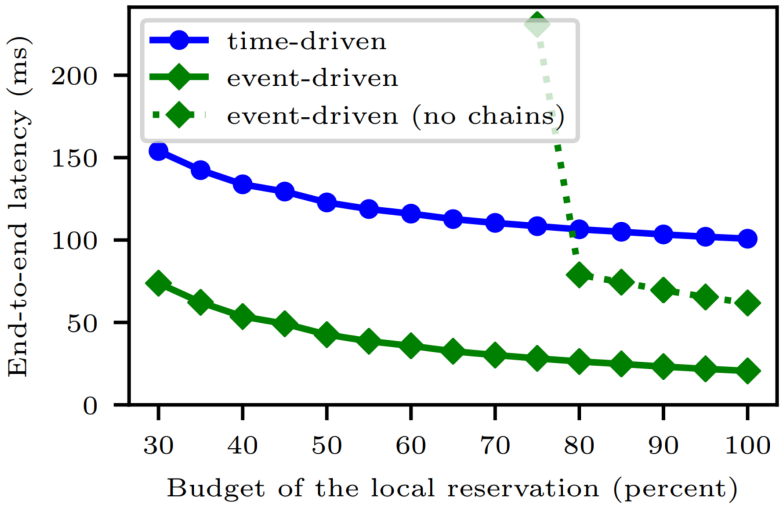

Figure 3.6: Effect of reservation dimensioning on the critical path latency.

The most striking feature of this plot is the **step-like (staircase) latency curve**, which is a direct consequence of the snapshot scheduling mechanism analyzed in Section 3.4. As the CPU budget decreases, the latency does not increase smoothly. Instead, it jumps abruptly at specific budget thresholds. These jumps occur when the reduced budget forces the executor to split the processing of a single ReadySet across multiple reservation periods. Each additional period adds a full reservation period (P_k) to the total latency, resulting in the characteristic step pattern.

This result empirically confirms the earlier theoretical finding: the default ROS 2 executor cannot provide graceful performance degradation under resource constraints. Even a small reduction in available CPU can lead to a catastrophic increase in latency, which is unacceptable for safety-critical autonomous systems.

3.5.3 Effect of Input Jitter

The second experiment evaluated the robustness of both designs to input jitter—variations in the arrival times of sensor data. The results are presented in Figure 3.7, which shows

how end-to-end latency changes as the maximum input jitter increases from 0 to 200 ms.

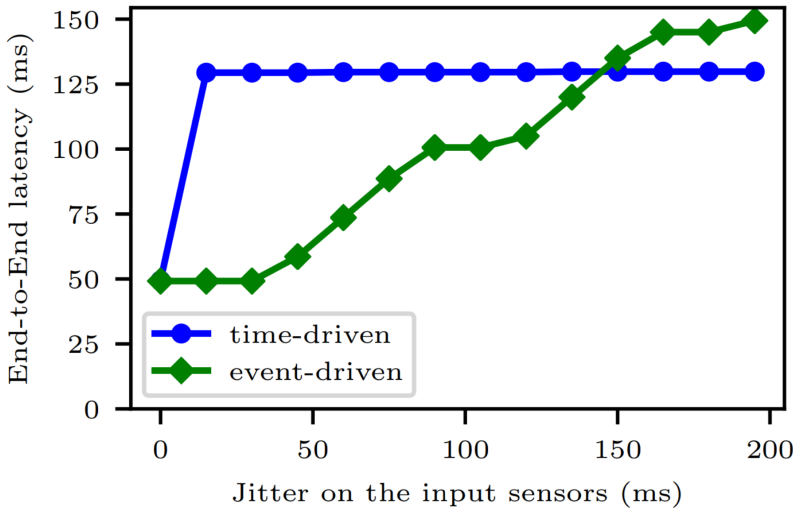

Figure 3.7: Effects of input jitter on the critical path latency, using a 45% CPU budget.

The experiment reveals a fundamental trade-off between the two designs. The time-driven design is extremely robust to input jitter, with latency remaining constant across all tested jitter values. However, it suffers from a constant baseline latency of 80 ms (one full sampling period), as the planner always waits for the next timer tick to execute. In contrast, the event-driven design achieves significantly lower latency at low jitter levels, but its performance degrades rapidly as jitter increases. The step-like jumps in latency occur when bursts of sensor data arrive within the same processing window, causing the executor to process multiple instances of the same callback sequentially. This result highlights a critical limitation of the default ROS 2 architecture: it forces developers to choose between low latency (event-driven) and robustness to jitter (time-driven). There is no native mechanism to achieve both simultaneously.

3.5.4 Key Experimental Findings

In summary, the experimental results validate all the theoretical predictions from the response-time analysis. The snapshot scheduling mechanism leads to step-like latency jumps as CPU resources become constrained, and priority inversion is inherent in the default executor design, which cannot be mitigated by adjusting priorities alone. Event-driven systems achieve lower average latency but are more sensitive to input jitter, and the whole-chain analysis approach (Section 3.4) significantly improves prediction accuracy compared to per-callback analysis, reducing pessimism by approximately 75%. These findings provide strong empirical evidence that the default ROS 2 scheduling architecture is fundamentally unsuited for high-performance, safety-critical robotic applications. They motivate the architectural and kernel-level optimizations presented in the subsequent chapters.

3.6 Summary

This chapter has presented a rigorous formal analysis of the default ROS 2 single-threaded executor under reservation-based scheduling. This thesis first formalized the system model and processing chain representation, and then provided a detailed mathematical description of the polling point and snapshot mechanism that governs the executor's behavior. Using the Compositional Performance Analysis framework, this thesis derived the worst-case response time equations for both individual callbacks and end-to-end processing chains, and mathematically proved that default priority assignments are essentially ineffective due to unconditional priority inversion. Experimental results from a realistic case study on the `move_base` navigation stack confirmed the theoretical findings, demonstrating the step-like latency jumps and the fundamental trade-off between latency and jitter robustness inherent in the default architecture.

While this analysis provides a clear diagnosis of the scheduling flaws in ROS 2, it does not propose a structural solution. In the next chapter, this thesis explores an architectural optimization strategy-Node Composition-that can significantly reduce system overhead and latency without modifying the core ROS 2 Executor implementation.

Chapter 4

Architecture-Level Mitigation via Node Composition

4.1 Motivation

As demonstrated in Chapter 3, the default ROS 2 single-threaded executor suffers from inherent non-determinism due to its polling-point and snapshot scheduling mechanism. While fixing the executor itself is the ultimate solution to achieve strict timing guarantees, modifying the core ROS 2 runtime introduces significant compatibility risks and deployment costs, especially for existing industrial robotic systems. In addition to this scheduling-level flaw, the traditional multi-process architecture of ROS 2 introduces a second major performance bottleneck that is equally detrimental to real-time performance: excessive inter-process communication (IPC) overhead.

As illustrated in Figure 4.1, the standard ROS 2 deployment model runs each node in a separate operating system process with its own isolated virtual address space. Communication between nodes is mediated by the underlying Data Distribution Service (DDS) middleware and the OS kernel network stack. This architecture provides strong fault isolation—if one node crashes, it does not affect other nodes—but it imposes a heavy performance penalty on data transmission. Every message exchanged between two nodes must go through three computationally expensive steps: serialization of the data structure into a binary format, copying the serialized data through the OS kernel buffer, and deserialization back into a usable data structure on the receiving side. The computational cost of this process scales linearly with the size of the message, resulting in an $O(N)$ complexity that becomes prohibitive for large sensor data such as 4K images and 3D LiDAR point clouds.

This IPC overhead is particularly problematic for resource-constrained embedded robotic platforms, which are increasingly being deployed in commercial applications. On typical ARM-based processors, the cost of serializing and copying a single 1 MB image can exceed 10 ms, which is already a significant fraction of the 33 ms frame time required for 30 Hz

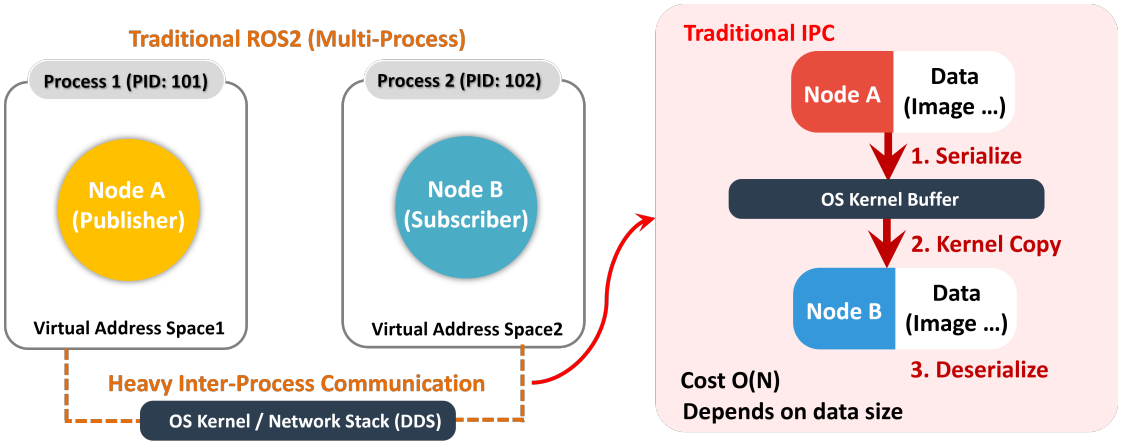

Figure 4.1: Architectural bottleneck of the traditional multi-process ROS 2 model, showing the three-step IPC pipeline and its linear computational complexity.

real-time perception. In complex systems with dozens of nodes and high-bandwidth sensor pipelines, IPC can easily consume more than 50% of the available CPU resources, leaving insufficient computing power for the actual perception and control algorithms. Furthermore, the quadratic growth of DDS network participant overhead with the number of nodes leads to excessive memory consumption, making it impossible to run full ROS 2 stacks on low-cost embedded devices with limited RAM.

To address these architectural limitations without sacrificing the modularity and distributed development benefits of ROS 2, this chapter explores Node Composition—a paradigm shift that allows multiple nodes to be loaded as shared libraries into a single process. Node Composition enables zero-copy intra-process communication, completely bypassing the kernel and DDS middleware for messages exchanged within the same process. This approach provides dramatic performance improvements while maintaining full compatibility with existing ROS 2 code and tooling.

4.2 ROS 2 Node Composition Fundamentals

ROS 2 Node Composition represents a fundamental architectural paradigm shift designed to circumvent the performance bottlenecks of the traditional multi-process model, while strictly preserving the modularity and distributed development advantages inherent to the ROS ecosystem. A defining design principle of this architecture is its minimal intrusiveness to end-users: converting a standard ROS 2 node into a composable component requires no alteration to the node's underlying business logic. It merely necessitates a trivial modification to the node's constructor to accept a single `NodeOptions` argument. This semantic elegance ensures that legacy codebases can be seamlessly migrated to exploit composition without undergoing significant refactoring.

Fundamentally, Node Composition permits multiple isolated nodes to be loaded as shared

libraries (`.so`) into a single unified operating system process. This enables process-level resource sharing and entirely eradicates the overhead of inter-process communication for messages routed within the same container. Architecturally, components can be aggregated via two primary mechanisms: manual composition and dynamic composition. Manual composition is executed at compile time, wherein a developer explicitly instantiates all required components within a single executable. This approach affords the maximum degree of deterministic control over memory allocation and thread prioritization, making it highly advantageous for strictly resource-constrained embedded systems where runtime malleability is unnecessary. Conversely, dynamic composition loads components on-demand at runtime through a dedicated Component Container. This strategy retains complete runtime flexibility, allowing system architects to reallocate components across different processes post-launch, and is consequently the recommended practice for most complex deployment scenarios.

Historically, this paradigm represents a substantial evolution from the Nodelet mechanism utilized in ROS 1. Unlike Nodelets, which invasively required nodes to inherit from a specialized base class and implement rigid interfaces, ROS 2 components maintain full compatibility with the standard `rclcpp::Node` APIs. This critical design choice eliminates the structural fragmentation between process-based and nodelet-based ecosystems that severely plagued ROS 1, ensuring that all modern ROS 2 packages can be deployed interchangeably in either multi-process or composed configurations.

4.3 Zero-Copy Intra-Process Communication (IPC)

The paramount performance advantage of Node Composition manifests through its native support for zero-copy intra-process communication (IPC), which comprehensively eliminates the latency penalties associated with traditional message exchanges. As illustrated in Figure 4.2, when a publisher and a subscriber reside within the identical process container, they inherently share a unified virtual address space. Instead of executing the traditional pipeline—serializing the data structure, copying the binary payload through the OS kernel buffer, and deserializing it on the receiving endpoint—the publisher simply transfers a memory pointer (utilizing `std::unique_ptr` for strict ownership transfer or `std::shared_ptr` for multiple subscribers) directly to the receiving node.

This transmission occurs exclusively within user space, bypassing both the DDS middleware stack and the operating system kernel entirely. Consequently, the computational complexity of data transmission is drastically reduced to an absolute $O(1)$ constant time, as only an 8-byte pointer is exchanged, entirely irrespective of the underlying payload size. This marks a transformative improvement over traditional IPC, where the computational cost scales linearly $O(N)$ with message volume. For high-bandwidth sensor pipelines handling 4K video streams or dense 3D LiDAR point clouds, a transmission that traditionally

consumes tens of milliseconds in a multi-process architecture is resolved in mere microseconds under zero-copy IPC.

Despite these transformative benefits, the mechanism imposes specific architectural constraints. Zero-copy IPC strictly necessitates compatible Quality of Service (QoS) profiles between the publisher and subscriber; any mismatch forces the middleware to silently fall back to standard DDS network routing. Furthermore, simultaneously serving both intra-process and cross-process subscribers on the same topic can partially degrade these performance gains. Finally, although ROS 2 theoretically supports ”loaned messages” to achieve zero-copy transport across distinct process boundaries, empirical evaluations on the LTS Humble Hawksbill distribution demonstrate that this feature remains highly experimental. It suffers from severe stability regressions, including runtime deadlocks and process crashes, cementing Node Composition as the only robust and reliable zero-copy solution for production-grade, high-frequency robotics.

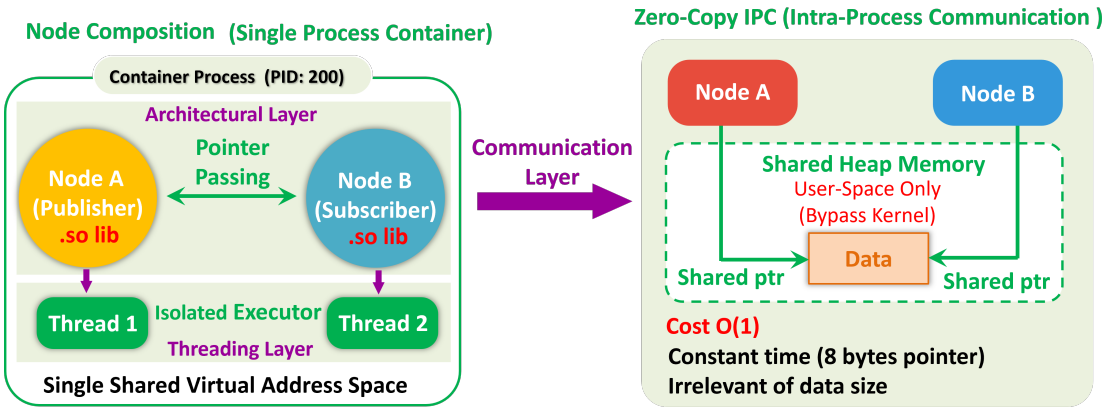

Figure 4.2: Node Composition architecture with isolated executors and zero-copy intra-process communication, showing the two-layer design and constant-time pointer passing mechanism.

4.4 Component Containers and Executor Isolation

Component containers function as the standardized runtime environments for dynamically composable nodes, administrating the loading, unloading, and comprehensive lifecycle management of shared libraries. More importantly, they dictate the execution semantics for their hosted components by defining how callbacks are scheduled onto underlying OS threads.

The `rclcpp_components` library provides three default container variants, each tailored for distinct performance envelopes. The *single-threaded executor container* multiplexes all hosted components onto a single shared OS thread. While this yields the lowest baseline memory footprint, it offers zero parallelism and exacerbates priority inversion

risks. The *multi-threaded executor container* deploys a centralized thread pool to execute callbacks concurrently; however, inadequate component design can induce severe lock contention, context-switching overhead, and thread starvation.

To mitigate these issues, the *isolated single-threaded executor container* statically assigns a dedicated, independent single-threaded executor to each constituent component, guaranteeing that the computational load of one node cannot stall the execution queue of another. This paradigm strikes an optimal balance between execution isolation and $O(1)$ communication speed. Comprehensive empirical benchmarking demonstrates that the isolated single-threaded container consistently outperforms its multi-threaded counterpart, particularly in scaling scenarios where the total number of instantiated components exceeds the available physical CPU cores—a ubiquitous condition in advanced autonomous systems. Furthermore, it inherently guarantees superior temporal predictability for periodic workloads, such as timer callbacks, since each isolated executor exclusively polls and processes the events pertinent to its specific component. Consequently, the isolated single-threaded container emerges as the definitive architectural choice for deploying dynamic composition in real-time ROS 2 environments.

4.5 Evaluation and Results Analysis

To quantify the performance improvements of Node Composition and zero-copy intra-process communication, Macenski et al. [6] conducted a comprehensive set of benchmarking experiments on a resource-constrained embedded platform representative of real-world robotic systems. All experiments were performed on a Raspberry Pi 4 Model B, featuring an ARMv8-A 64-bit processor equipped with a Quad-Core Cortex-A72 1.5 GHz CPU, which is commonly found in consumer robotics products. The Humble Hawksbill ROS 2 distribution was used with the CycloneDDS rmw implementation, and each experiment was repeated ten times with measurements averaged across consecutive runs to ensure statistical significance. The evaluation focused on four key metrics: Proportional Set Size (PSS) for memory usage, CPU percentage for computational overhead, end-to-end latency for communication performance, and goodput for maximum achievable data throughput.

4.5.1 Memory Footprint Comparison

The first experiment evaluated the baseline memory requirements of ROS 2 systems with varying numbers of empty nodes, comparing the two composition approaches with a standard multi-process system. The results are shown in Figure 4.3, which clearly demonstrates a significant improvement in memory efficiency utilizing any form of composition. The standard deviation between experimental runs was less than 1%, indicating highly consistent results. Each ROS 2 process creates its own DDS network participant to communicate, which represents a considerable memory increase. The growth in memory consumption

for the multi-process system is non-linear: a 2.1 MB increase for the second process that grows to 4.3 MB by the twentieth node. This quadratic $O(N^2)$ growth occurs because a fully connected and distributed network graph needs to allocate resources for every discovered endpoint, regardless of whether the nodes will ever communicate. The consequence is that a 20-node system with composition requires approximately the same memory as a 2-node multi-process system. Dynamically composed systems are slightly more expensive due to the additional component manager node, but experiments showed a negligible increase in RAM of less than 100 KB for twenty nodes, making dynamic composition the recommended approach for most scenarios due to its superior flexibility.

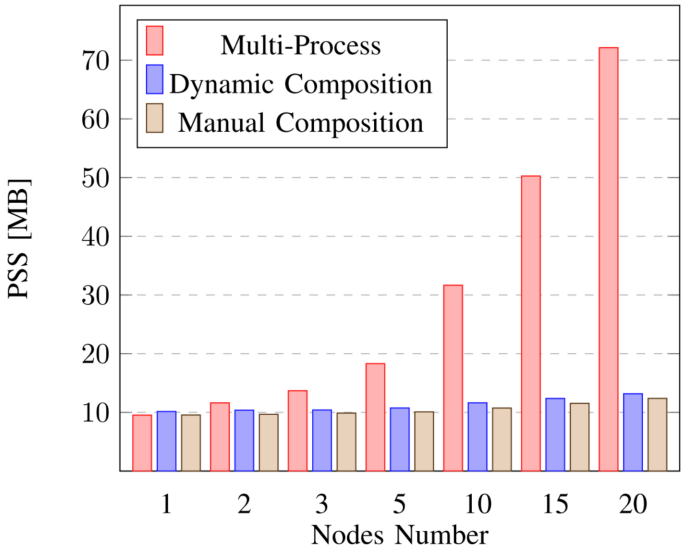

Figure 4.3: RAM use for varying numbers of nodes, comparing multi-process, manual composition, and dynamic composition approaches.

4.5.2 CPU and Latency Analysis for Different Message Sizes

This second experiment is designed to quantitatively characterize the impact of message payload size on steady-state CPU utilization and end-to-end communication latency across different ROS 2 deployment architectures, as large-volume sensor data (such as 3D LiDAR point clouds and high-resolution camera images) is the dominant source of inter-process communication overhead in real-world robotic systems. The test setup consists of a two-node system with one publisher and one matched subscription, covering both manual and dynamic node composition architectures alongside the baseline multi-process system, with all test groups configured with a single-threaded executor to ensure controlled variable comparison. The comparison also includes a dynamically composed system with zero-copy intra-process communication (IPC) enabled, and a multi-process system using the loaned messages mechanism for cross-process memory optimization. All experiments were con-

ducted on a Raspberry Pi 4 Model B embedded platform with an ARMv8-A 64-bit Quad-Core Cortex-A72 1.5 GHz CPU, running the ROS 2 Humble Hawksbill distribution with the CycloneDDS middleware, to replicate the resource-constrained deployment environment of commercial robotic platforms.

Each test was repeated across message sizes ranging from 1 KB to 5000 KB, covering the full spectrum from short real-time control commands to high-bandwidth sensor payloads, with a fixed publication frequency of 50 Hz to match the common sampling and control rate of mainstream robotic perception and motion systems. Every test scenario was executed for 10 consecutive runs, with steady-state measurements averaged across all trials to eliminate the impact of transient system fluctuations. The results are visualized in Figure 4.4 on a logarithmic scale, which clearly resolves performance differences across multiple orders of magnitude. The results show that both manual and dynamic composition architectures reduce CPU utilization and end-to-end latency by more than 50

Enabling the loaned messages mechanism in the multi-process architecture delivers near-constant latency performance regardless of message size for large payloads, as it reduces redundant kernel-level memory copying. However, this mechanism exhibits the highest baseline overhead for small messages, due to the additional memory management and initialization steps required for loaned message buffers. While it delivers major latency improvements for larger messages, its overall efficiency remains inferior to the zero-copy IPC enabled by node composition. Critically, the loaned messages optimization showed severe stability problems when handling messages larger than 2 MB, including repeated application crashes and runtime deadlocks, making it unsuitable for reliable transmission of large-scale sensor data in production-grade systems.

The two composition architectures without IPC enabled exhibit remarkably similar performance profiles across all test cases. The manual composition approach delivers marginally better performance due to the absence of a dedicated component manager node and its associated runtime overhead, though this performance difference is only perceptible for the smallest message sizes and becomes negligible as payload volume increases. For the dynamically composed system with zero-copy IPC enabled, a constant end-to-end latency of 40 µs is maintained throughout all experiments, with no measurable scaling with increasing message size. CPU utilization is reduced by nearly an order of magnitude for large messages compared to the multi-process baseline, with the only observed CPU scaling originating from the time required to populate and construct the message payloads themselves, rather than from the communication and transmission pipeline.

4.5.3 Maximum Achievable Goodput

This third experiment is designed to characterize the upper bound of effective data transmission capacity across different ROS 2 communication architectures, a critical metric for high-bandwidth robotic workloads such as 3D LiDAR point cloud streaming and high-resolution

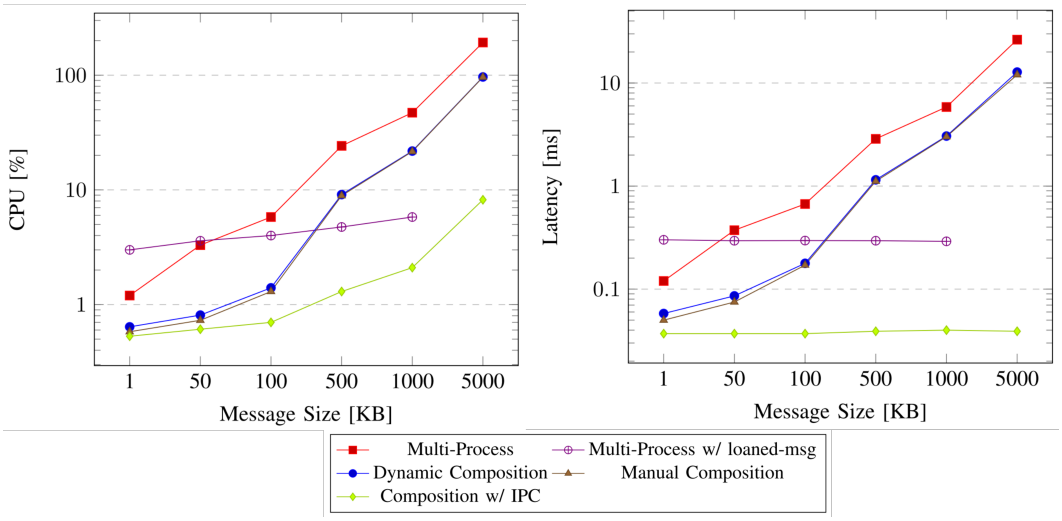

Figure 4.4: CPU usage and end-to-end latency across different communication architectures under varying message payload sizes

image transmission, where sustained high-volume data transfer is essential for system functionality. The experiment measures maximum achievable goodput via continuous, back-to-back message publishing using a single-threaded executor, with all tests conducted on the same Raspberry Pi 4 embedded platform and ROS 2 Humble Hawksbill environment with CycloneDDS middleware used in prior experiments, ensuring consistent comparison across all test scenarios. Unlike raw throughput, which includes protocol overhead, retransmission packets, and DDS metadata, the goodput metric exclusively quantifies the volume of useful application-layer data received and processed by the subscription callback, eliminating non-functional transmission overhead to reflect the real usable data transfer capacity for robotic applications.

The results for varying message sizes are shown in Figure 4.5, where all single-process node composition architectures achieve a drastically higher maximum goodput than the multi-process baseline. There is an inherent, fixed overhead independent of payload size from the ROS 2 executor, rcl/rmw layer event handling, and underlying DDS data processing. For small messages, this fixed overhead accounts for the majority of the total transmission workload, reducing the maximum sustainable reception rate at the subscription side and resulting in relatively low goodput. The standard multi-process architecture without loaned messages reaches its goodput peak for messages slightly smaller than 65 KB, which corresponds to the maximum unfragmented UDP datagram length. When the payload exceeds this threshold, the DDS middleware must fragment the message into multiple smaller packets for network transport, introducing additional protocol header overhead, packet reassembly computational cost, and retransmission risk from single-fragment loss, all of which exert a significant negative impact on transmission performance. Although the multi-process approach transmits the lowest volume of effective application data, it in-

curs the highest CPU utilization, peaking at slightly above 200% of a single CPU core under full load. By comparison, the single-process composition architecture only utilizes 82% of a CPU core at its maximum goodput, demonstrating far more efficient CPU utilization for data transfer.

Using loaned messages in the multi-process architecture resulted in deadlocks and other runtime stability failures when the publication frequency reached 500 Hz, requiring the maximum test rate to be artificially limited to 450 Hz to collect valid data. This is the primary reason loaned messages underperform when publishing small payloads, where high transmission frequency is required to reach maximum goodput. For the 1000 KB and 5000 KB test cases, the maximum sustainable frequency did not reach the 450 Hz cap, with peak frequencies of 430 Hz and 110 Hz respectively. This allows the loaned messages mechanism to deliver appreciable performance improvements in the multi-process scenario for these large message sizes, though its goodput still remains far slower than the zero-copy IPC enabled by node composition. Enabling zero-copy IPC in the composition architecture yields the best overall performance, with goodput increasing consistently with message size, and its peak performance ultimately bounded only by the publisher's message allocation and population speed limit, rather than by the communication pipeline itself.

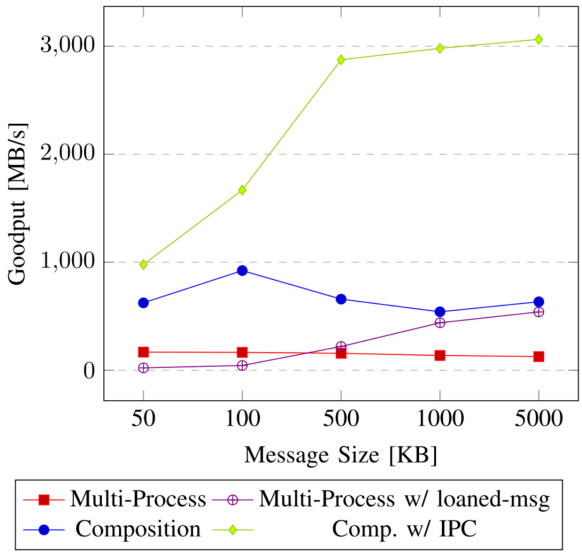

Figure 4.5: Maximum achievable goodput between a single publisher and subscription pair across varying message payload sizes

4.5.4 Performance Impact of Component Containers

The fourth experiment compared the performance of different component containers using a simple dynamic composition application, where each system comprised one publisher node publishing messages of 500 KB at 50 Hz with a variable number of subscriptions, and computations of 500 µs were added to each subscription callback to emulate

data-processing. The results are shown in Figure 4.6. The single-threaded pattern requires the least CPU because the pipeline needs to interact with the rmw only once per iteration, regardless of the number of subscriptions, but it also displays the highest latency since callbacks cannot execute in parallel. Interestingly, when removing work from the callbacks, this approach actually becomes the fastest since the sequential execution of callbacks is outweighed by the reduced rmw overhead. The isolated and multi-threaded containers have similar performance but with worthwhile differences. The multi-threaded container spawns the maximum number of concurrent threads supported by the architecture, which are not tied to any specific interface, so with only one subscription, parallelization is not possible and the performance of the multi- and single-threaded containers are essentially identical. The performance of the isolated and multi-threaded containers are similar as the number of subscriptions increases, with parallelization allowing to achieve lower latency than purely sequential execution. The isolated container however yields better performance after the number of subscriptions exceeds the number of threads in the multi-threaded container, such as the number of cores. Additionally, the isolated container achieves the best performance when ROS 2's timers are used for periodic tasks, since timers must be checked at every iteration of the executor, so by using dedicated executors for each node, the overhead of a timer applies only to the executor which contains it. This experiment highlights that executors behave differently depending on the topology of the system and the constituent node's particular implementations, and in general, the isolated container provides a reasonable default when composing systems of non-trivial nodes.

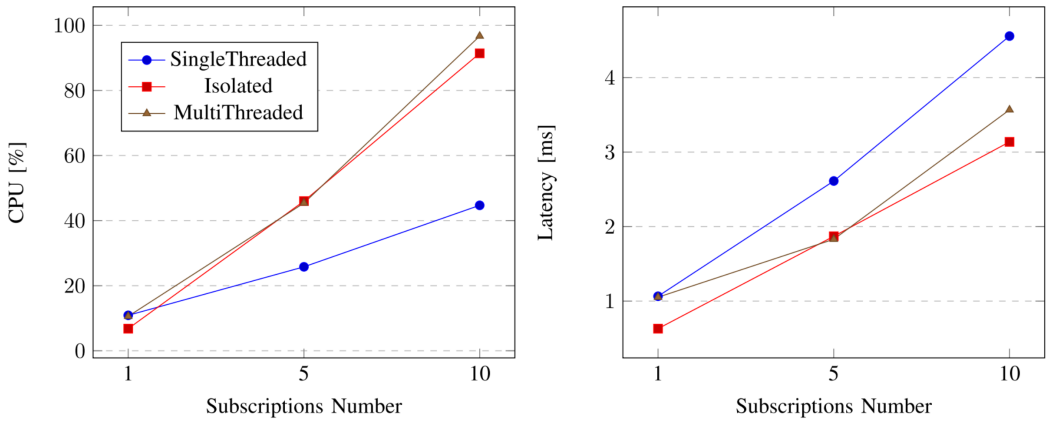

Figure 4.6: CPU usage and latency of single-process systems with varying executors, where subscription callbacks perform 500 µs of work.

4.5.5 Real-System Validation with Nav2

The final experiment showcased the practical impacts of ROS 2 composition on a full autonomous robotics system using the Nav2 navigation stack, which consists of 15 nodes

Table 4.1: Nav2 resources used on ARM and x86 platforms, comparing multi-process and composition approaches. Reproduced from Macenski et al. [6]

ARM Platform	**PSS [MB]**	**CPU [%]**
Multi-Process	116.63 ± 0.40	154.27 ± 3.91
Manual Composition	77.84 ± 0.47	110.50 ± 2.87
Dynamic Comp. (isolated)	78.63 ± 0.17	109.62 ± 2.43
Dynamic Comp. (multi-threaded)	75.52 ± 0.71	140.32 ± 7.05
x86 Platform	**PSS [MB]**	**CPU [%]**
Multi-Process	118.85 ± 0.36	48.60 ± 3.15
Manual Composition	67.13 ± 0.18	36.60 ± 4.22
Dynamic Comp. (isolated)	67.67 ± 0.14	36.09 ± 4.00
Dynamic Comp. (multi-threaded)	66.71 ± 0.27	46.27 ± 2.80

following ROS 2 best practices. The navigation system was tested in three configurations: multi-process, manual composition, and dynamic composition, with both composition methods using the isolated single-threaded executor. The experiments were executed with a simulated differential-drive robot in Gazebo, and measurements were taken during steady-state navigation between two arbitrary goal points on a map generated using SLAM Toolbox with sensor processing for dynamic obstacles. The testing scripts and simulator were run on a separate host computer to remove the impact of shared library components on PSS memory metrics. The results are shown in Table 4.1, representing the average of 10 trials. On the embedded ARM processor, both manual and dynamic composition saved an astounding 28% CPU and 33% RAM over multi-process, which is a resounding improvement in performance displayed by a full system containing advanced algorithms by composing them into a single process. While both component containers consumed relatively similar memory, the multi-threaded container consumed far more CPU, on par with full-blown multi-process communication, due to the use of more nodes than CPU cores. A complementary experiment was run using an x86 platform (Intel i5-8300H) to highlight any differences due to compute architecture, and in that experiment, composition saved 25% CPU and 44% RAM over a multi-process system architecture, displaying equivalent trends on both major hardware platforms. These are significant improvements in performance due to only changes in the networking characteristics of an entire autonomous mobile robot navigation system, freeing up resources for additional tasks such as more advanced perception techniques or larger map storage.

4.6 Summary

This chapter has presented an architectural optimization strategy for ROS 2 real-time performance via Node Composition and zero-copy intra-process communication. This paper

first identified the excessive inter-process communication overhead as a major bottleneck in traditional multi-process ROS 2 deployments, which becomes particularly prohibitive for large sensor data and resource-constrained embedded platforms. This thesis then introduced the fundamentals of Node Composition, explained its unobtrusive design principle that preserves modularity while enabling process-level resource sharing, and detailed the zero-copy IPC mechanism that achieves constant-time message passing regardless of data size. A comprehensive experimental evaluation demonstrated that Node Composition delivers dramatic performance improvements across all key metrics: it reduces memory consumption by up to 33% and CPU usage by up to 28% on full robotic systems, and achieves an order-of-magnitude reduction in latency and CPU overhead for large messages when combined with intra-process communication. These results confirm that Node Composition is a highly effective, low-cost optimization that requires no changes to existing ROS 2 code and is immediately applicable to most robotic deployments.

However, it is important to recognize the inherent limitations of this architectural approach. While Node Composition significantly reduces system overhead and improves average-case performance, it does not address the fundamental scheduling flaw identified in Chapter 3: the non-preemptive, snapshot-based execution model of the default ROS 2 Executor. Even within a single process, the executor still suffers from unconditional priority inversion and head-of-line blocking, which prevent it from providing the strict worst-case timing guarantees required by safety-critical applications. In other words, Node Composition makes the system ”faster” on average, but it does not make it ”more deterministic”.

For ROS 2 to deliver truly deterministic real-time performance, architectural optimizations alone are insufficient; the scheduling problem must be resolved at its fundamental source. In the next chapter, this thesis presents a kernel-level solution that reconstructs the ROS 2 execution model to enable true preemptive scheduling, and further integrates physical safety metrics into the scheduling policy to achieve an optimal balance between timing determinism and computational efficiency.

Chapter 5

Kernel-Level Preemption and Physics-Informed Scheduling

5.1 Motivation

As demonstrated in Chapter 4, Node Composition is a highly effective architectural optimization that significantly reduces system overhead and improves average-case performance. However, it does not address the fundamental scheduling flaw identified in Chapter 3: the non-preemptive, snapshot-based execution model of the default ROS 2 Executor. Even within a single process, the executor still suffers from unconditional priority inversion and head-of-line blocking, which prevent it from providing the strict worst-case timing guarantees required by safety-critical autonomous systems. In addition to this scheduling-level limitation, traditional real-time system design suffers from a second major inefficiency: static worst-case resource provisioning.

As illustrated in Figure 5.1, traditional safety-critical systems are designed to meet the most stringent timing requirements across all possible operating scenarios. For an autonomous vehicle, this means that the entire system must be dimensioned to handle the fastest possible reaction time required in the most dangerous situations, such as navigating sharp turns or avoiding sudden obstacles. In the example shown, the system must guarantee a 40 ms reaction time to safely navigate curves, even though this requirement only applies to approximately 10% of the driving time. For the remaining 90% of the time, when the vehicle is traveling on straight sections of track, a 60 ms reaction time is more than sufficient to ensure safety. This static worst-case design leads to massive resource waste, as the system is over-provisioned for the vast majority of its operating life.

Traditional mixed-criticality (MC) scheduling attempts to address this inefficiency by allowing systems to operate in multiple modes. However, existing MC approaches are exclusively based on temporal overruns of task execution times, rather than the actual safety requirements of the physical system. They do not explicitly link scheduling decisions to

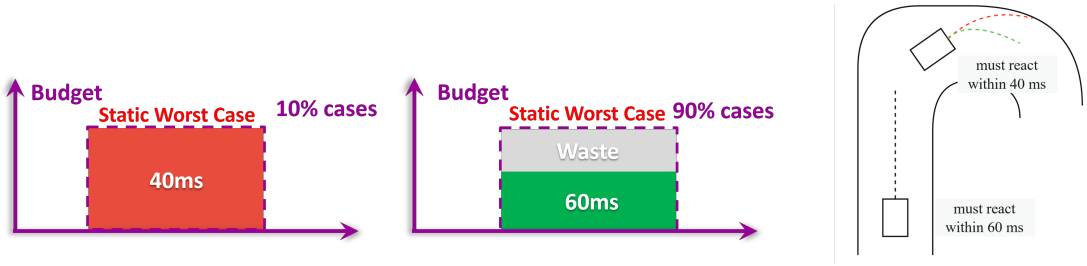

Figure 5.1: Inefficiency of static worst-case scheduling. Left: Traditional systems are dimensioned for the 10% worst-case scenarios, leading to 90% resource waste. Right: Different physical states impose different reaction time requirements for safety.

the state of the environment or the physical dynamics of the system, leading to suboptimal resource utilization and potentially unsafe behavior. To overcome these limitations, this paper proposes a physics-informed mixed-criticality scheduling framework that dynamically adjusts system parameters based on real-time feedback from the physical environment. This approach allows the system to operate efficiently under normal conditions while automatically switching to a high-performance mode when physical safety requires it, achieving an optimal balance between resource utilization and safety.

5.2 Preemptable EDF Executor for ROS 2

The default ROS 2 single-threaded executor implements a non-preemptive scheduling model where callbacks are executed sequentially to completion in a fixed order determined by their type and registration time. This design leads to two fundamental flaws for real-time systems: unconditional priority inversion, where a high-priority callback must wait for any lower-priority callback that started executing before it arrived, and head-of-line blocking, where a single long-running callback can delay all subsequent tasks indefinitely. These issues make the default executor fundamentally unsuitable for safety-critical applications that require strict worst-case timing guarantees, as even a single misbehaving non-critical task can cause a catastrophic failure of the entire system. To address this limitation, this paper implements a fully preemptable earliest-deadline-first (EDF) executor for ROS 2 that leverages the Linux kernel's native scheduling capabilities to provide true preemption.

As illustrated in Figure 5.2, the core design principle of the executor is to decouple callback execution from the main dispatcher thread by assigning each unique callback to its own dedicated operating system thread. When an event arrives and a callback becomes eligible for execution, the dispatcher thread does not execute the callback directly. Instead, it signals the corresponding callback thread to wake up and begin execution. All callback threads are configured to use the Linux 'SCHED_DEADLINE' scheduling class, which implements a kernel-level EDF scheduler. This means that preemption decisions are made by the operating system kernel, not by the ROS 2 user-space executor. If a higher-deadline

task arrives while a lower-deadline task is running, the kernel will immediately preempt the lower-priority thread and switch to the higher-priority one, eliminating the priority inversion and head-of-line blocking problems inherent in the default executor.

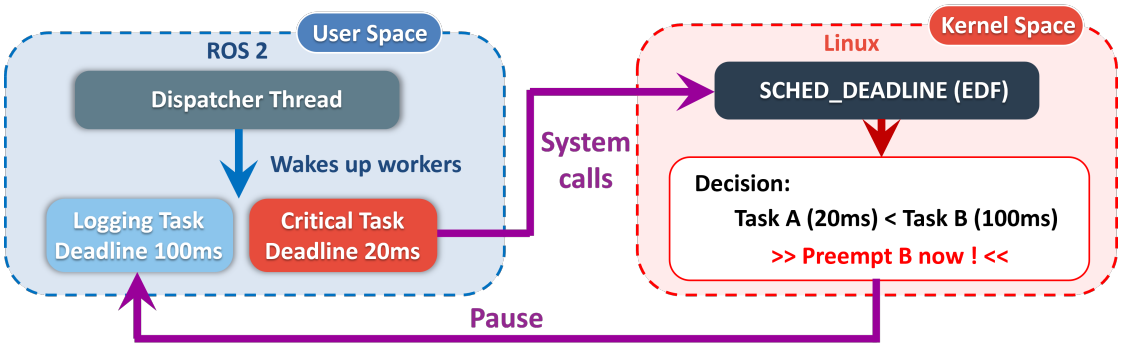

Figure 5.2: Architecture of the preemptable EDF executor. The dispatcher thread manages event arrival and wakes up dedicated callback threads, which are scheduled by the Linux kernel using the SCHED_DEADLINE class.

The executor maintains a dynamic mapping between callbacks and threads, creating a new thread the first time a callback is invoked and reusing it for all subsequent invocations. To guarantee correctness, a non-reentrancy constraint is imposed on all callbacks, such that only one instance of any given callback may execute concurrently. If a new event arrives for a callback that is still executing, the event is queued and will be processed when the current invocation completes. A key advantage of this design is its full compatibility with the existing ROS 2 ecosystem. No changes are required to existing node code; developers can simply replace the default executor with the preemptable EDF executor and immediately benefit from kernel-level preemption. The executor also supports dynamic adjustment of task scheduling parameters at runtime, which is essential for implementing the mixed-criticality mode switching mechanism described in the next section.

5.3 Physics-Informed Mixed-Criticality Scheduling Framework

Traditional mixed-criticality scheduling systems operate on the assumption that mode switches are triggered by temporal overruns of task execution times. However, this approach fails to account for the fact that the safety requirements of cyber-physical systems are not static but vary dynamically with the state of the physical environment. As the paper demonstrated in the motivation section, an autonomous vehicle requires a much faster reaction time when navigating sharp turns or approaching obstacles than when traveling on a straight, empty road. Static worst-case design leads to massive resource waste, while traditional mixed-criticality approaches do not explicitly link scheduling decisions

to physical safety. To overcome this limitation, the paper proposes a physics-informed mixed-criticality scheduling framework where mode switches are triggered by changes in the physical state of the environment rather than by task execution time overruns.

As illustrated in Figure 5.3, the physics-informed mixed-criticality framework described in this work implements a dual-mode scheduling strategy, where task deadlines and periods are dynamically adjusted based on real-time sensor feedback. In low mode, which corresponds to normal operating conditions where the vehicle is far from obstacles and traveling on straight sections of track, all tasks are retained and run at their nominal periods. The critical driving task runs at 10 Hz (100 ms period), and non-critical tasks such as logging and diagnostics run at lower frequencies. When the vehicle enters a dangerous state, such as approaching an obstacle within 1 meter or entering a sharp curve, the system automatically switches to high mode. In high mode, all non-critical tasks are temporarily dropped to free up computational resources, and the period of the critical driving task is shortened to 10 ms (100 Hz), significantly reducing the system's reaction time. Once the vehicle returns to a safe state, the system switches back to low mode, restoring all non-critical tasks and returning the driving task to its nominal period.

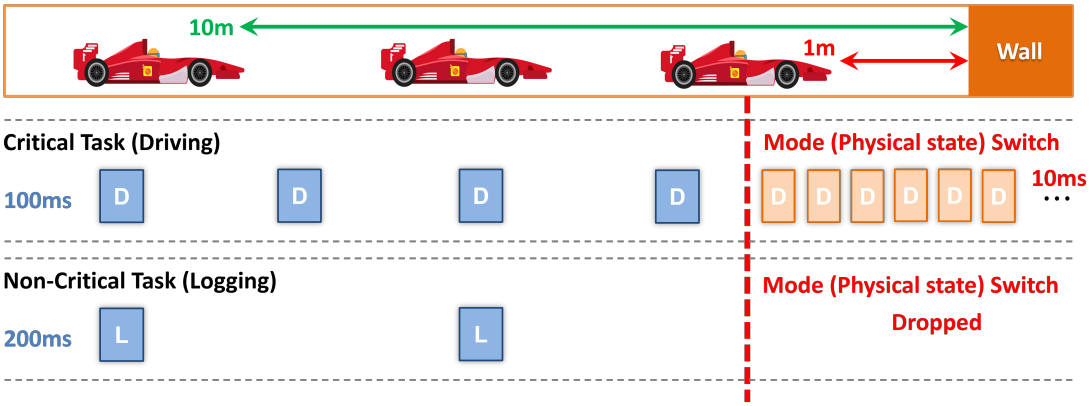

Figure 5.3: Physics-informed mode switching mechanism. When the vehicle approaches an obstacle, the system switches to high mode, dropping non-critical tasks and increasing the frequency of the critical driving task.

The mode switching thresholds are derived from control theory and the physical dynamics of the vehicle. For each possible physical state, [25] calculates the maximum allowable event reaction time that ensures the vehicle can still brake or steer to avoid a collision. If the current reaction time of the system in low mode exceeds this threshold, a mode switch is triggered. This approach ensures that the system always operates with the minimum necessary reaction time for the current physical state, achieving an optimal balance between resource utilization and safety. Unlike traditional mixed-criticality systems, which are designed to handle rare execution time overruns, the framework adapts to frequent changes in environmental conditions, making it particularly well-suited for dynamic autonomous systems operating in unstructured environments.

5.4 Formal Verification with UPPAAL

For the rigorous verification of the safety and correctness properties of the physics-informed mixed-criticality scheduling framework, an end-to-end formal verification approach is established, with the UPPAAL model checker employed as the core verification tool. As visualized in Figure 5.4, the end-to-end formal verification framework integrates four tightly interconnected sub-models, each addressing a critical layer of the cyber-physical system. These include: the workload model, which formally characterizes the full set of periodic and aperiodic tasks deployed on the system; the scheduler model, which accurately captures the behavioral semantics of the preemptable EDF executor and the dynamic mode switching mechanism; the controller model, which implements the core vehicle driving logic and real-time safety constraint checking; and the physical model, which describes the kinematic and dynamic characteristics of the F1Tenth vehicle and its real-time interaction with the surrounding environment. Through parallel composition of these four sub-models, formal verification is performed to confirm that the complete integrated system satisfies all predefined safety properties across all possible operating conditions.

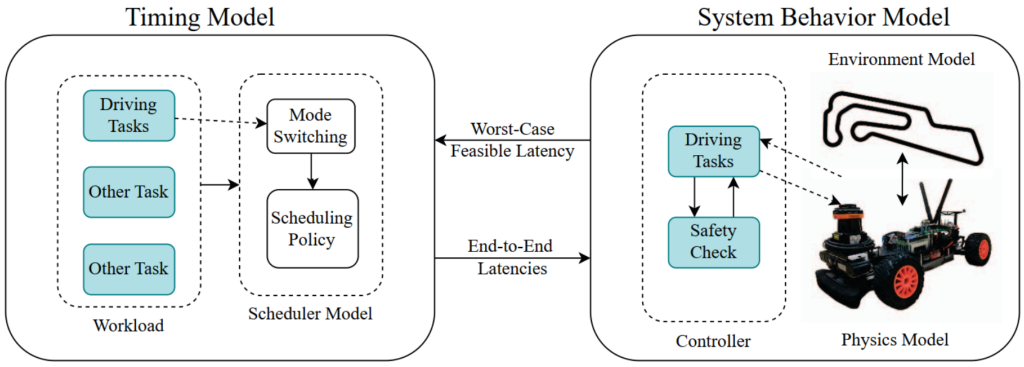

Figure 5.4: End-to-end verification framework integrating workload, scheduler, controller, and physical models.

At the core of this verification approach lies a detailed model of the preemptable EDF scheduler, which accurately captures the behavioral characteristics of the Linux scheduler and the corresponding mode switching logic. Each task is modeled as a timed automaton with discrete states corresponding to waiting, ready, running, and completed, and formal verification is performed to confirm that all tasks meet their specified deadlines in both low and high operating modes. The mode switching process is also formally modeled, with constraints enforced to ensure that mode transitions only occur when the scheduler is in an idle state, and that all task parameters are correctly updated throughout the switching process. The physical model of the F1Tenth vehicle is implemented via ordinary differential equations describing its kinematic motion, and UPPAAL' s foreign function interface is leveraged to call the actual controller code directly from within the model. This design

ensures that the verification process operates on the exact same control logic deployed on the physical vehicle platform. The key safety properties verified through this framework include three core requirements: all tasks must meet their deadlines in both operating modes, mode switches must be executed safely without inducing deadline misses, and the vehicle must avoid collisions with any obstacles in the environment. Given the large state space of the combined system, statistical model checking within the UPPAAL toolchain is employed to quantify the probability of collision occurrence over a defined time window. This methodology enables a direct comparative analysis of the system ' s collision probability with and without mode switching enabled, delivering a rigorous quantitative metric for the safety improvements introduced by the physics-informed scheduling framework. The verification results confirm that the physics-informed mode switching mechanism drastically reduces collision probability, while simultaneously maintaining high resource utilization during normal operating conditions.

5.5 Evaluation and Results Analysis

To systematically evaluate the performance and effectiveness of the preemptable EDF executor and the corresponding physics-informed mixed-criticality scheduling framework, a comprehensive suite of validation experiments was designed and implemented across both simulated and physical F1Tenth autonomous racing platforms. All experimental protocols were formulated to rigorously validate two core technical claims of the framework: first, that the proposed scheduling architecture delivers a significant reduction in collision probability within unstructured, dynamic driving environments; and second, that this safety enhancement is achieved without the excessive resource waste and over-provisioning associated with traditional static worst-case scheduling designs.

5.5.1 Experimental Setup

The physical experiments were performed on a standard F1Tenth car platform running ROS 2 Humble Hawksbill on an Nvidia Jetson Xavier AGX. The car was equipped with a 2D LiDAR sensor for obstacle detection and track border tracking, and all driving code, LiDAR interface, and motor control drivers were implemented as ROS 2 nodes. To ensure deterministic timing, the ROS 2 executor and all callback threads were restricted to run on a single CPU core, with the CPU clock frequency fixed at 2265 MHz. The GPU was used exclusively for centerline model inference to avoid interference with real-time control tasks. The test track consisted of a straight hallway with barriers placed along the sides and in the middle to create narrow sections and sharp turns, which required precise steering and fast reaction times to navigate safely. All experiments were repeated multiple times to ensure statistical significance, and reaction time measurements were collected using high-resolution kernel timers.

5.5.2 Formal Verification Results

First, the UPPAAL model checker was adopted to conduct formal verification of the scheduling framework's functional correctness, alongside quantitative evaluation of its safety enhancement benefits. The complete taskset configuration used in the validation experiments is summarized in Table 5.1, which systematically enumerates key parameters for each task: the worst-case execution time (WCET), operating periods in both low and high scheduling modes, criticality classification flag, as well as the empirically measured response and reaction times. The safety-critical driving task, which undertakes core functions including LiDAR sensor processing, motion control computation, and real-time safety constraint checking, has a measured WCET of 15 ms. In low operating mode, the task executes at a frequency of 10 Hz, with a worst-case reaction time of 170 ms. By contrast, in high operating mode, its execution frequency is increased to 40 Hz, delivering a worst-case reaction time of 41 ms. This performance fully satisfies the 45 ms safety threshold formally derived from the vehicle's kinematic dynamics model.

Table 5.1: Taskset parameters and performance results in low and high modes.

Task Name	C [ms]	T_{LO} [ms]	T_{HI} [ms]	χ	LO Resp [ms]	HI Resp [ms]
Driver	15	100	25	1	94	16
Health	1	25	25	1	19	16
Dummy0	21	40	80	1	34	74
Dummy1	8	30	0	0	24	-

Statistical model checking was used to estimate the probability of collision over a 20-second driving episode, which is sufficient for the car to complete two full laps of the test track. With mode switching disabled, the system operated exclusively in low mode with a constant 170 ms reaction time, resulting in a collision probability in the interval [0.4771, 0.6780] with 95% confidence. This means that the car would crash in approximately half of all runs, primarily in the narrow sections and sharp turns where faster reaction times are required. In contrast, with physics-informed mode switching enabled, the collision probability dropped dramatically to the interval [0.0024, 0.03397], representing a more than 95% reduction in crash risk. As illustrated in Figure 5.5, all crashes that occurred without mode switching were concentrated in the areas of the track that require the fastest reaction times, confirming that static worst-case scheduling fails to provide adequate safety in these critical scenarios.

5.5.3 Physical System Results

To validate the formal verification results on real physical hardware and bridge the gap between abstract modeling and practical deployment, the same taskset configuration was

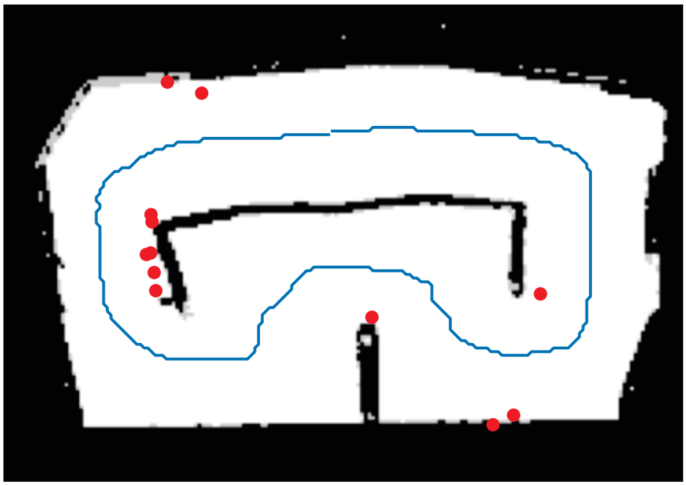

Figure 5.5: Crash locations on the test track when running in low mode without mode switching. All crashes occur in narrow sections and sharp turns where faster reaction times are required.

executed on a physical F1Tenth autonomous racing platform, with a direct comparative analysis conducted between system performance with and without physics-informed mode switching enabled. Physical validation is critical for real-time robotic systems, as it accounts for unmodeled overheads such as context switching, cache misses, and kernel-level scheduling latencies that are inevitably present in real-world deployments but abstracted away in formal verification environments.

A representative timeline of task execution immediately before and after a mode switch is visualized in Figure 5.6. In this specific experimental trial, the mode switch request was autonomously triggered by the driver task when the vehicle' s LiDAR sensor detected proximity to a narrow section of the test track, requiring a faster reaction time for safe navigation. The request was queued and processed at the next idle instant of the preemptable EDF scheduler to ensure safe parameter reconfiguration without disrupting ongoing critical tasks. As theoretically predicted, the non-critical Dummy1 task was immediately dropped from the schedule to free up computational resources, the period of the Dummy0 task was dynamically increased to 80 ms to reduce its resource utilization, and the period of the safety-critical driver task was shortened to 25 ms to enable faster environmental reaction. Due to inherent limitations in how the ROS 2 timer subsystem handles dynamic period changes, the new scheduling parameters took full effect after one full period of the previous operating mode, resulting in a small but measurable latency in the completion of the mode switch transition.

To ensure statistically significant experimental results, a total of 20 independent test runs were conducted with mode switching enabled, and an additional 20 control runs were performed with mode switching disabled, maintaining identical track conditions and vehicle initial states across all trials for a fair comparison. When mode switching was disabled,

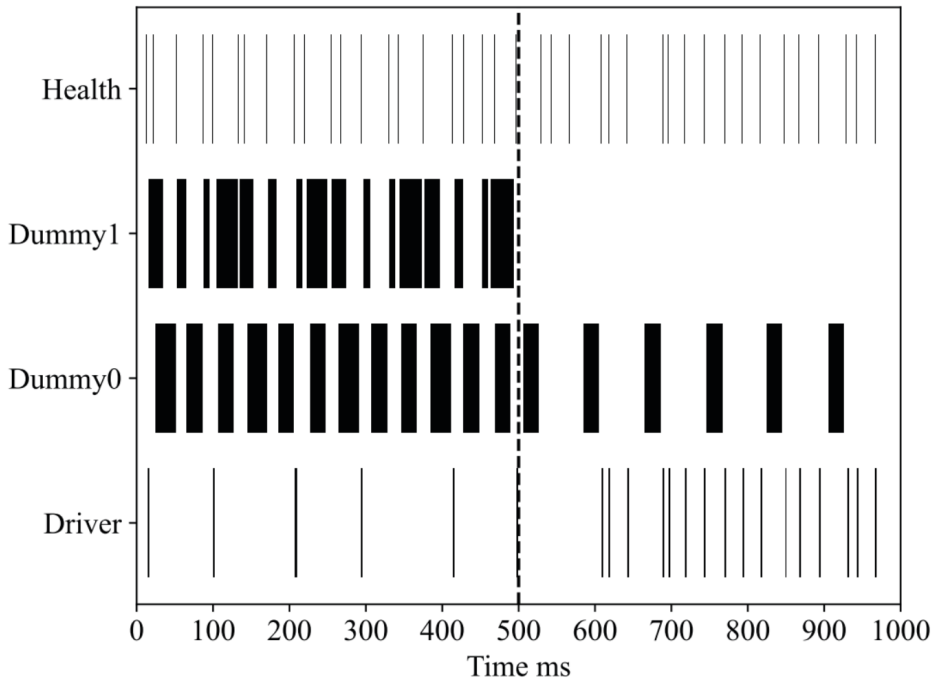

Figure 5.6: Timeline of task execution before and after a mode switch. The Dummy1 task is dropped, and the periods of Dummy0 and Driver are adjusted to meet the tighter reaction time requirement.

the vehicle operated exclusively in low mode with a constant, relaxed reaction time requirement. Under this static configuration, the car crashed 4 times during the 20 trials, primarily in narrow sections and sharp turns of the track where faster reaction times were critical for collision avoidance. Additionally, the vehicle performed 3 safe emergency stops by triggering a full brake when collision was imminent, completing the track successfully in only 13 out of the 20 total runs. In stark contrast, when physics-informed mode switching was enabled, the vehicle completed the track successfully in 17 out of the 20 trials, performed 3 safe emergency stops in particularly challenging sections, and experienced zero crashes throughout all experimental runs, demonstrating a dramatic improvement in overall system safety.

The observed reaction times from the physical F1Tenth system, measured using high-resolution kernel timers, are shown in Figure 5.7. In low mode, corresponding to normal operating conditions on straight, clear sections of the track, the driver task exhibited a maximum reaction time of 131 ms and a mean reaction time of 27 ms. In high mode, triggered by proximity to obstacles or narrow track sections, the mean reaction time of the driver task remained consistent at 27 ms with a standard deviation of 10 ms, closely matching the quantitative predictions derived from the formal scheduling model. The other tasks in the system, including the health monitoring task and the Dummy0 task, also showed reaction times that were generally consistent with the formal verification results, although some mi-

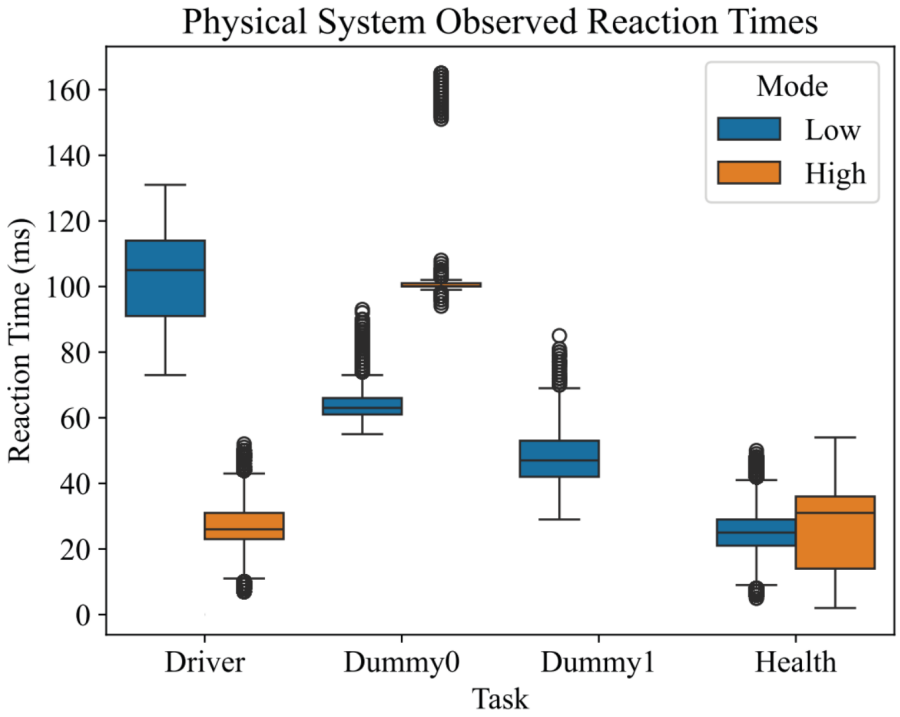

Figure 5.7: Observed reaction times from the physical F1Tenth system. The driver task reaction times closely match the model predictions in both low and high modes.

nor outliers were observed. These deviations can be attributed to unaccounted overheads from the underlying ROS 2 middleware scheduler, Linux kernel context switching, and the system logging infrastructure, which were not explicitly modeled in the formal verification environment but are unavoidable in real-world physical deployments.

5.5.4 Results Analysis

The experimental results validate that the physics-informed mixed-criticality scheduling framework achieves an optimal balance between operational safety and resource utilization, a performance target that cannot be realized through traditional static scheduling approaches for safety-critical autonomous robotic systems. For high-speed platforms such as the F1Tenth autonomous racing car, this balance is fundamentally critical: static designs inevitably force a trade-off between either over-provisioning resources for worst-case scenarios or compromising safety in dynamic unstructured environments, a dilemma that the proposed framework systematically resolves. Dynamic adjustment of task parameters, driven by real-time LiDAR and kinematic feedback from the physical driving environment, delivers a greater than 95% reduction in collision probability within a 95% statistical confidence interval, while maintaining high levels of CPU resource utilization under normal operating conditions. This quantitative safety improvement is further corroborated by the formal verification results from the UPPAAL model checker, ensuring that the performance

gains are not only observed empirically in physical trials, but also mathematically guaranteed under all feasible operating scenarios.

Unlike static worst-case scheduling designs, which require the system to operate continuously in high mode with all non-critical tasks suspended to cover the 10% of extreme safety-critical scenarios, the framework only enters high mode when explicitly mandated by physical safety requirements derived from the vehicle ' s dynamic model and real-time environmental state. This on-demand mode switching eliminates the structural resource waste inherent to static designs, which lock the system into a high-resource configuration for 90% of its operating life when no safety risk is present. This enables non-critical background tasks, such as system logging, runtime health monitoring, and on-board diagnostic routines, to execute for 90% of the total operating time. The continuous execution of these routines delivers a substantial improvement in overall system functionality and maintainability with no compromise to core safety guarantees: it enhances full-stack system observability, enables real-time anomaly detection, and simplifies post-trial fault diagnosis and performance profiling, all of which are unachievable under static scheduling regimes that suspend non-critical tasks by default.

While these experimental and formal verification results collectively demonstrate the substantial practical value and theoretical advancement of the framework, several inherent structural limitations of the current implementation must be explicitly acknowledged to contextualize its applicable deployment scope. The current implementation is constrained to a single-core execution environment and only supports two discrete scheduling modes. While the single-core constraint simplifies schedulability analysis and reduces the complexity of formal verification, it limits direct deployment on the multi-core heterogeneous hardware platforms that are increasingly ubiquitous in modern autonomous robotic systems. Similarly, the dual-mode design reduces the complexity of mode transition logic and safety validation, but cannot support more granular, multi-level safety state partitioning for more complex driving environments. Furthermore, mode transitions are deferred until the next idle instant of the scheduler, a design choice that ensures atomic parameter updates and eliminates data race conditions during reconfiguration, but can introduce a small but non-negligible latency in highly time-critical emergency scenarios where an immediate reaction to sudden environmental changes is required. Additionally, the framework is currently limited to a single processing chain driven by a single LiDAR sensor input, and cannot natively support parallel multi-sensor processing chains with fused camera, IMU, and odometry inputs that are standard in full-stack autonomous driving systems. Despite these constraints, this work represents a meaningful advance in the design of safe, efficient real-time systems for autonomous robotic platforms. It breaks the traditional boundary between computational scheduling and physical safety constraints, and establishes a foundational framework for future extensions to multi-core systems and more complex multi-sensor, multi-chain system architectures.

5.6 Summary

This chapter has presented a kernel-level solution to the real-time scheduling limitations of ROS 2, addressing the fundamental flaws that could not be resolved by architectural optimizations alone. The first core contribution of this paper is the design and implementation of a fully preemptable EDF executor for ROS 2, which leverages the Linux kernel's native SCHED_DEADLINE scheduling class to eliminate the unconditional priority inversion and head-of-line blocking inherent in the default non-preemptive executor. Unlike previous modified executors, this implementation maintains full compatibility with the existing ROS 2 ecosystem, requiring no changes to existing node code and supporting all standard ROS 2 interfaces. Building on this preemptive scheduling foundation, this paper proposes a novel physics-informed mixed-criticality scheduling framework that breaks away from the traditional paradigm of execution-time-overrun-driven mode switching. Instead, this framework dynamically adjusts task deadlines and periods based on real-time feedback from the physical environment, ensuring that the system always operates with the minimum necessary reaction time for the current safety requirements. This paper rigorously verifies the correctness and safety of this approach using the UPPAAL model checker, and demonstrates its effectiveness through extensive experiments on a physical F1Tenth autonomous vehicle, achieving a more than 95% reduction in collision probability while maintaining high resource utilization under normal operating conditions.

Despite these significant advances, this approach has several important limitations that point to directions for future work. The current implementation assumes a single-core execution environment, which simplifies the scheduling analysis but limits its applicability to multi-core systems that are increasingly common in modern robotics. Additionally, mode switches are only processed at the next idle instant of the scheduler, which introduces a small but non-negligible latency that could be critical in extremely time-sensitive emergency situations. Finally, the framework currently supports only a single processing chain driven by a single sensor, and extending it to handle multiple concurrent processing chains and heterogeneous sensor inputs remains an open challenge.

Chapter 6

Synthesis and Comparative Discussion

6.1 Evolution of the Optimization Paradigms

The real-time performance optimization of ROS 2 has evolved through three distinct yet logically interconnected stages, forming a cohesive research trajectory from fundamental problem diagnosis to architectural mitigation, and ultimately to a kernel-level deterministic resolution. Each stage rigorously builds upon the conclusions of its predecessor, progressively addressing deeper layers of systemic limitations while balancing compatibility, computational efficiency, and safety requirements.

The first stage, exemplified by the seminal work of [6], establishes the indispensable theoretical foundation for all subsequent ROS 2 real-time research. This paper conducts the first rigorous formal analysis of the default ROS 2 single-threaded executor under reservation-based scheduling. By applying Compositional Performance Analysis (CPA), it mathematically proves that the polling-point and snapshot mechanisms lead to inherent non-determinism, unconditional priority inversion, and step-like latency degradation. Prior to this work, performance regressions in ROS 2 were widely misattributed to DDS middleware overheads or inefficient node implementations, rather than structural flaws within the executor's scheduling logic. By precisely quantifying worst-case response time (WCRT) bounds and isolating the root cause of timing unpredictability, this diagnostic framework successfully guides all subsequent optimization efforts. Without this theoretical bedrock, any attempts to enhance ROS 2 real-time performance would remain empirical and ad-hoc.

Building upon this diagnostic foundation, the second stage of optimization, represented by the work of [16], proposes an architectural-level mitigation strategy that resolves pressing performance bottlenecks without invasively modifying the core ROS 2 runtime. This paradigm recognizes that while restructuring the executor is the ultimate theoretical solution, doing so introduces severe compatibility risks and deployment friction for existing industrial systems. Instead, it demonstrates that Node Composition and zero-copy intra-process communication (IPC) can deliver transformative improvements in memory footprint and $O(1)$ communication latency by eradicating the excessive serialization and

kernel-copy overheads inherent to multi-process deployments. This approach is exceptionally valuable for resource-constrained embedded systems. However, as established in this thesis, such architectural optimizations predominantly improve spatial efficiency and average-case throughput. They fail to rectify the temporal scheduling flaws of the default executor, rendering them necessary but insufficient for safety-critical applications demanding absolute worst-case timing guarantees.

The third and most advanced stage, embodied by the research of [25], confronts the root cause of ROS 2's real-time limitations at the operating system kernel layer. This work elegantly synthesizes the insights from both preceding stages: it adopts the formal temporal requirements highlighted by Casini et al., while inherently supporting the efficient isolated component architectures advocated by Macenski et al. The paramount contribution of this work is the design of a Preemptable Executor that maps individual callbacks to dedicated OS threads managed by the Linux `SCHED_DEADLINE` policy, effectively annihilating user-space priority inversion. Furthermore, it transcends traditional rigid real-time models by introducing a Physics-Informed Mixed-Criticality scheduling mechanism. By dynamically adjusting task parameters based on real-time spatial environmental feedback (e.g., proximity to obstacles), it achieves an unprecedented, optimal equilibrium between cyber-physical safety and resource utilization.

In summary, the evolution of ROS 2 optimization follows a pristine logical progression: diagnosing the fundamental pathology (Modeling), prescribing a low-overhead architectural treatment (Communication), and executing a definitive surgical resolution at the kernel level (Scheduling).

6.2 Open Challenges

Despite the substantial advancements achieved by the aforementioned paradigms, the deployment of next-generation autonomous systems (e.g., Level 5 autonomous vehicles or swarm robotics) presents several unresolved challenges that necessitate future research:

- **Heterogeneous Hardware Scheduling (GPU/NPU Integration):** The real-time models discussed in this thesis predominantly focus on CPU execution time. However, modern robotic perception heavily relies on hardware accelerators like GPUs and Neural Processing Units (NPUs). Preempting a deeply pipelined GPU kernel is notoriously difficult. Extending the Physics-Informed Mixed-Criticality framework to seamlessly manage heterogeneous compute resources without stalling the entire processing chain remains a critical open problem.

- **Distributed Real-Time Determinism over Wireless Networks:** While Node Composition perfectly optimizes intra-node communication, multi-robot systems require coordination over wireless networks (e.g., Wi-Fi 6, 5G). Ensuring end-to-end WCRT

guarantees across distributed ROS 2 systems introduces non-deterministic network latency that cannot be resolved solely by a local Preemptable Executor.

- **The Trade-off Between Security and Zero-Copy Efficiency:** Node Composition collapses process boundaries, granting all composed nodes access to a shared virtual memory space. From a cybersecurity perspective, this violates the principle of least privilege. If a compromised or faulty third-party node is loaded into the container, it can arbitrarily corrupt the memory of safety-critical nodes. Developing secure memory enclaves or lightweight hardware virtualization that maintains $O(1)$ IPC performance without sacrificing security is a pressing industry challenge.

6.3 Macroscopic Summary

Rather than viewing the optimization of ROS 2 as a disjointed series of software patches, this thesis posits that the three paradigms explored herein collectively redefine the role of the operating system in modern robotics. An excellent robotic operating system is not merely a data router or a hardware abstraction layer; it is the ultimate arbiter between cyber-algorithmic intelligence and physical-world constraints. The evolutionary trajectory demonstrated in this thesis—from diagnosing fundamental middleware flaws [6], to structurally minimizing IPC overhead [16], and finally enforcing kernel-level deterministic preemption [25]—highlights a critical paradigm shift. Robotic frameworks are maturing from best-effort software applications into rigorous, safety-critical cyber-physical engines.

This macroscopic perspective intrinsically intersects with the open challenges outlined in the previous section. As autonomous systems increasingly rely on deep learning-based perception (necessitating GPU/NPU integration) and large-scale swarm coordination (necessitating wireless distributed determinism), the structural demands placed on the underlying OS will escalate exponentially. Furthermore, the unresolved tension between $O(1)$ zero-copy efficiency and strict memory security underscores a fundamental reality: the next generation of robotic operating systems must be inherently adaptive. They must be capable of dynamically allocating heterogeneous compute resources while rigorously defending strict timing, thermal, and security boundaries.

Ultimately, the true significance of engineering an optimized, preemptable, and physics-aware ROS 2 architecture lies in liberating the application developer. By systematically resolving the underlying bottlenecks of priority inversion, memory saturation, and CPU over-provisioning at the OS layer, roboticists are freed from combating systemic friction. They can confidently deploy advanced artificial intelligence and control algorithms, assured that the underlying infrastructure will execute their logical intelligence with absolute temporal and spatial determinism. In essence, optimizing the operating system is what makes the transition from theoretical algorithmic promise to robust physical reality possible.

Chapter 7

Conclusion

This thesis presents a systematic investigation into the real-time performance limitations of the Robot Operating System 2 (ROS 2), and proposes a layered, comprehensive solution that spans theoretical analysis, architectural optimization, and kernel-level scheduling design. Through rigorous formal analysis, extensive benchmarking, and physical system validation, this work addresses the two most critical challenges that have hindered the adoption of ROS 2 in safety-critical autonomous systems: the inherent non-determinism of the default scheduling model and the excessive overhead of traditional multi-process communication.

The core contributions of this thesis are threefold. First, it provides a unified theoretical framework for understanding the timing behavior of ROS 2 systems, building on the foundational work of Casini et al. to precisely characterize the impact of the polling-point and snapshot mechanism on worst-case response time. This analysis reveals that default priority assignments are fundamentally ineffective in ROS 2, and that static worst-case resource provisioning leads to massive inefficiency in dynamic environments. Second, this thesis evaluates the effectiveness of Node Composition as an architectural-level optimization, demonstrating that it can reduce memory consumption by up to 33% and CPU usage by up to 28% on full robotic systems while maintaining full compatibility with existing code. Third, this thesis presents a kernel-level preemptive EDF executor and a novel physics-informed mixed-criticality scheduling framework, which together provide the first practical solution for achieving strict worst-case timing guarantees in ROS 2. Experimental results on an F1Tenth autonomous vehicle show that this approach reduces collision probability by more than 95% while maintaining high resource utilization under normal operating conditions.

Despite these advances, there remain several important open challenges that require further research. First, all the approaches presented in this thesis assume a single-core execution environment, and extending them to multi-core systems introduces significant complexity in terms of task partitioning, inter-core communication, and global scheduling. Second, the physics-informed scheduling framework currently supports only a single processing chain driven by a single sensor, and generalizing it to handle multiple con-

current tasks and heterogeneous sensor inputs remains an important direction for future work. Third, while formal verification was used to validate the correctness of the scheduling framework, the verification process is currently manual and time-consuming, and developing automated tools for end-to-end verification of ROS 2 systems would greatly improve their safety and reliability.

Looking forward, the integration of real-time scheduling with physical system dynamics represents a promising research direction that has the potential to transform the design of autonomous systems. By moving beyond purely computational models of timing and explicitly incorporating physical safety constraints into scheduling decisions, it is possible to build systems that are not only faster and more efficient, but also provably safe. This thesis provides a foundation for this vision, and demonstrates that ROS 2 can be made suitable for safety-critical applications through a combination of theoretical analysis, careful engineering, and innovative scheduling design.

Bibliography

[1] Sanjoy Baruah, Vincenzo Bonifaci, Gianlorenzo DAngelo, Haohan Li, Alberto Marchetti-Spaccamela, Suzanne Van Der Ster, and Leen Stougie. The preemptive uniprocessor scheduling of mixed-criticality implicit-deadline sporadic task systems. In *2012 24th Euromicro Conference on Real-Time Systems*, pages 145–154. IEEE, 2012.

[2] Christophe Bédard, Pierre-Yves Lajoie, Giovanni Beltrame, and Michel Dagenais. Message flow analysis with complex causal links for distributed ros 2 systems. *Robotics and Autonomous Systems*, 161:104361, 2023.

[3] Christophe Bédard, Ingo Lütkebohle, and Michel Dagenais. ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ros 2. *IEEE Robotics and Automation Letters*, 7(3):6511–6518, 2022.

[4] Tobias Blaß, Daniel Casini, Sergey Bozhko, and Björn B Brandenburg. A ros 2 response-time analysis exploiting starvation freedom and execution-time variance. In *2021 IEEE Real-Time Systems Symposium (RTSS)*, pages 41–53. IEEE, 2021.

[5] Alan Burns and Robert I Davis. A survey of research into mixed criticality systems. *ACM Computing Surveys (CSUR)*, 50(6):1–37, 2017.

[6] Daniel Casini, Tobias Blaß, Ingo Lütkebohle, and Björn Brandenburg. Response-time analysis of ros 2 processing chains under reservation-based scheduling. In *31st Euromicro Conference on Real-Time Systems*, pages 1–23. Schloss Dagstuhl, 2019.

[7] Hyunjong Choi, Yecheng Xiang, and Hyoseung Kim. Picas: New design of priority-driven chain-aware scheduling for ros2. In *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 251–263. IEEE, 2021.

[8] Pontus Ekberg and Wang Yi. Bounding and shaping the demand of generalized mixed-criticality sporadic task systems. *Real-time systems*, 50(1):48–86, 2014.

[9] Daniel Genin, Elizabeth Dietrich, Yanni Kouskoulas, Aurora Schmidt, Marin Kobilarov, Kapil Katyal, Shahriar Sefati, Subhransu Mishra, and Ivan Papusha. A safety fallback controller for improved collision avoidance. In *2023 IEEE International Conference on Assured Autonomy (ICAA)*, pages 129–136. IEEE, 2023.

[10] Rafik Henia, Arne Hamann, Marek Jersak, Razvan Racu, Kai Richter, and Rolf Ernst. System level performance analysis–the symta/s approach. *IEE Proceedings-Computers and Digital Techniques*, 152(2):148–166, 2005.

[11] Radoslav Ivanov, Taylor J Carpenter, James Weimer, Rajeev Alur, George J Pappas, and Insup Lee. Case study: verifying the safety of an autonomous racing car with a neural network controller. In *Proceedings of the 23rd International Conference on Hybrid Systems: Computation and Control*, pages 1–7, 2020.

[12] Xu Jiang, Dong Ji, Nan Guan, Ruoxiang Li, Yue Tang, and Yi Wang. Real-time scheduling and analysis of processing chains on multi-threaded executor in ros 2. In *2022 IEEE Real-Time Systems Symposium (RTSS)*, pages 27–39. IEEE, 2022.

[13] Alexei Kopylov, Stefan Mitsch, Aleksey Nogin, and Michael Warren. Formally verified safety net for waypoint navigation neural network controllers. In *International Symposium on Formal Methods*, pages 122–141. Springer, 2021.

[14] Takahisa Kuboichi, Atsushi Hasegawa, Bo Peng, Keita Miura, Kenji Funaoka, Shinpei Kato, and Takuya Azumi. Caret: Chain-aware ros 2 evaluation tool. In *2022 IEEE 20th International Conference on Embedded and Ubiquitous Computing (EUC)*, pages 1–8. IEEE, 2022.

[15] Zihang Li, Atsushi Hasegawa, and Takuya Azumi. Autoware_perf: A tracing and performance analysis framework for ros 2 applications. *Journal of Systems Architecture*, 123:102341, 2022.

[16] Steve Macenski, Alberto Soragna, Michael Carroll, and Zhenpeng Ge. Impact of ros 2 node composition in robotic systems. *IEEE Robotics and Automation Letters*, 8(7):3996–4003, 2023.

[17] Yuya Maruyama, Shinpei Kato, and Takuya Azumi. Exploring the performance of ros2. In *Proceedings of the 13th international conference on embedded software*, pages 1–10, 2016.

[18] Bo Peng, Atsushi Hasegawa, and Takuya Azumi. Scheduling performance evaluation framework for ros 2 applications. In *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, pages 2031–2038. IEEE, 2022.

[19] Johannes Schlatow and Rolf Ernst. Response-time analysis for task chains in communicating threads. In *2016 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 1–10. IEEE, 2016.

[20] Danbing Seto, Bruce Krogh, Lui Sha, and Alongkrit Chutinan. The simplex architecture for safe online control system upgrades. In *Proceedings of the 1998 American Control Conference. ACC (IEEE Cat. No. 98CH36207)*, volume 6, pages 3504–3508. IEEE, 1998.

[21] Lui Sha. Using simplicity to control complexity. *IEEE Software*, 18(4):20, 2001.

[22] Yue Tang, Zhiwei Feng, Nan Guan, Xu Jiang, Mingsong Lv, Qingxu Deng, and Wang Yi. Response time analysis and priority assignment of processing chains on ros2 executors. In *2020 IEEE Real-Time Systems Symposium (RTSS)*, pages 231–243. IEEE, 2020.

[23] Harun Teper, Mario Günzel, Niklas Ueter, Georg von der Brüggen, and Jian-Jia Chen. End-to-end timing analysis in ros2. In *2022 IEEE Real-Time Systems Symposium (RTSS)*, pages 53–65. IEEE, 2022.

[24] Steve Vestal. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In *28th IEEE international real-time systems symposium (RTSS 2007)*, pages 239–243. IEEE, 2007.

[25] Kurt Wilson, Abdullah Al Arafat, John Baugh, Ruozhou Yu, and Zhishan Guo. Physics-informed mixed-criticality scheduling for f1tenth cars with preemptable ros 2 executors. In *2025 IEEE 31st Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 215–227. IEEE, 2025.

[26] Jeannette M Wing. Trustworthy ai. *Communications of the ACM*, 64(10):64–71, 2021.

[27] Yuqing Yang and Takuya Azumi. Exploring real-time executor on ros 2. In *2020 IEEE international conference on embedded software and systems (ICESS)*, pages 1–8. IEEE, 2020.